

中山大學
SUN YAT-SEN UNIVERSITY



国家超级计算广州中心
NATIONAL SUPERCOMPUTER CENTER IN GUANGZHOU

CUDA执行模型

任课教师：吴迪

Roadmap

- Introducing the CUDA Execution Model
- Understanding the Nature of Warp Execution
- Exposing Parallelism
- Avoiding Branch Divergence
- Unrolling Loops
- Dynamic Parallelism

参考阅读：Cheng-Ch3

1. Introducing the CUDA Execution Model

CUDA Execution Model

- In general, an execution model provides an operational view of how instructions are executed on a specific computing architecture.

- The CUDA execution model exposes an abstract view of the GPU parallel architecture, allowing you to reason about thread concurrency.

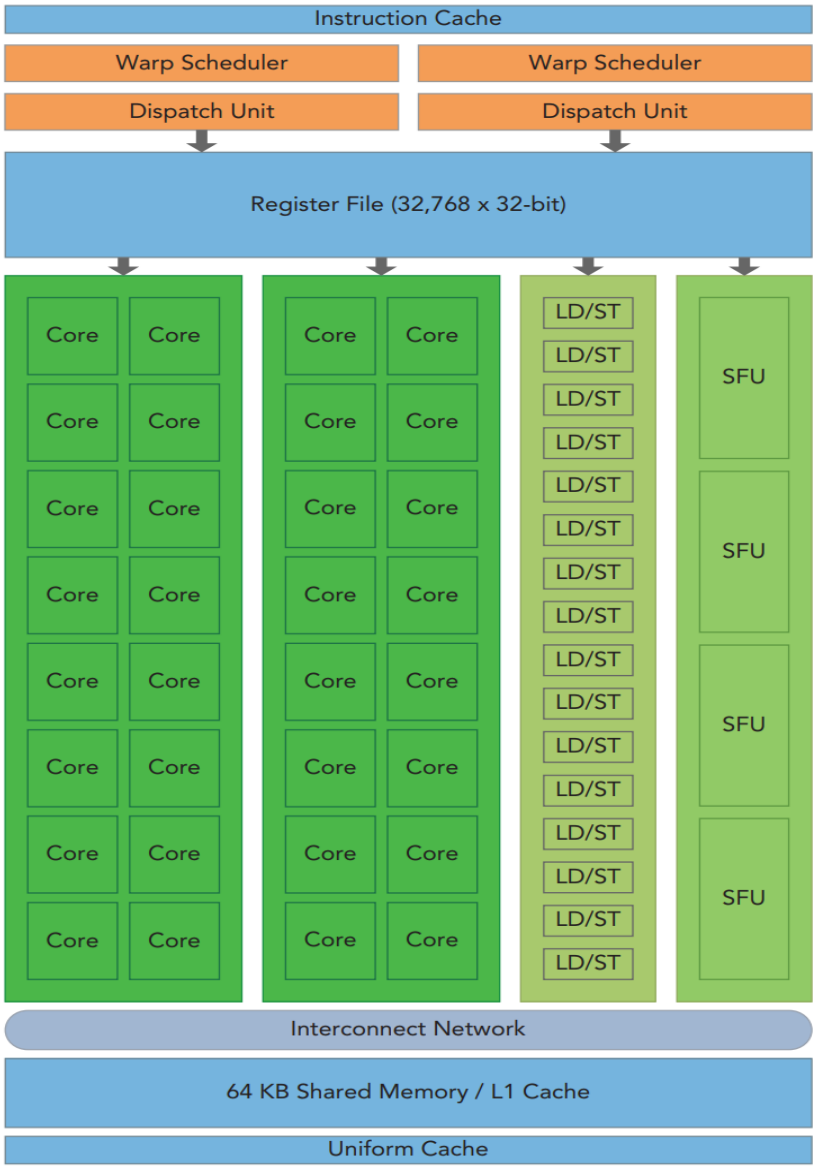
- CUDA execution model provides insights that are useful for writing efficient code
  - in terms of both instruction throughput and memory accesses.

1.1 GPU Architecture Overview

GPU Architecture Overview

- The GPU architecture is built around a scalable **array** of **Streaming Multiprocessors (SM).**

- **GPU hardware parallelism** is achieved through the replication of this architectural building block (SM)



Streaming Multiprocessors (SM)

- Each SM in a GPU is designed to support concurrent execution of hundreds of threads.
  - When a kernel grid is launched, the thread blocks of that kernel grid are distributed among available SMs for execution.
  - Once scheduled on an SM, the threads of a thread block execute concurrently only on that assigned SM.

- Multiple thread blocks may be assigned to the same SM at once and are scheduled based on the availability of SM resources.

- Instructions within a single thread are pipelined to leverage instruction-level parallelism

Warp

- CUDA employs a **Single Instruction Multiple Thread (SIMT)** architecture to manage and execute threads in groups of 32 called **warps**.
  - All threads in a warp execute the same instruction at the same time.
  - Each thread has its own instruction address counter and register state, and carries out the current instruction on its own data.

- Each SM partitions the thread blocks assigned to it into 32-thread warps that it then schedules for execution.

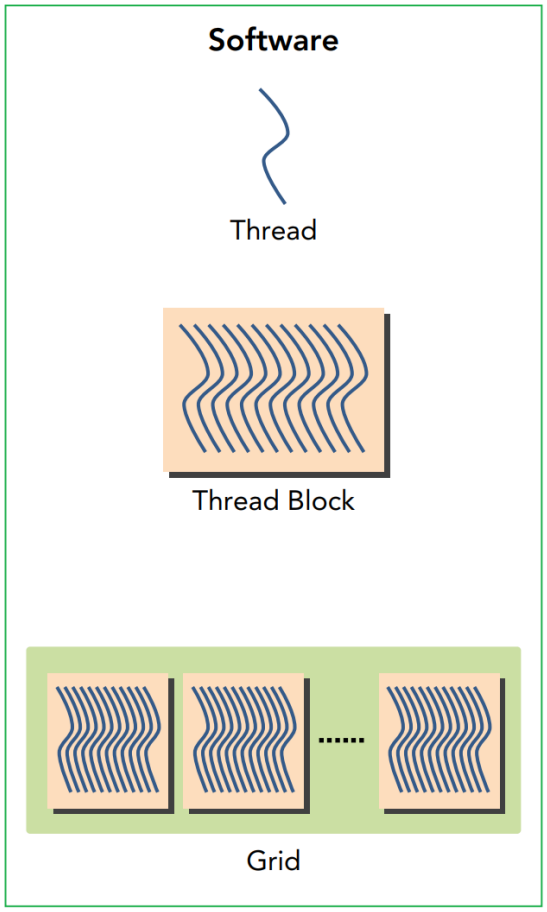
SIMT vs. SIMD

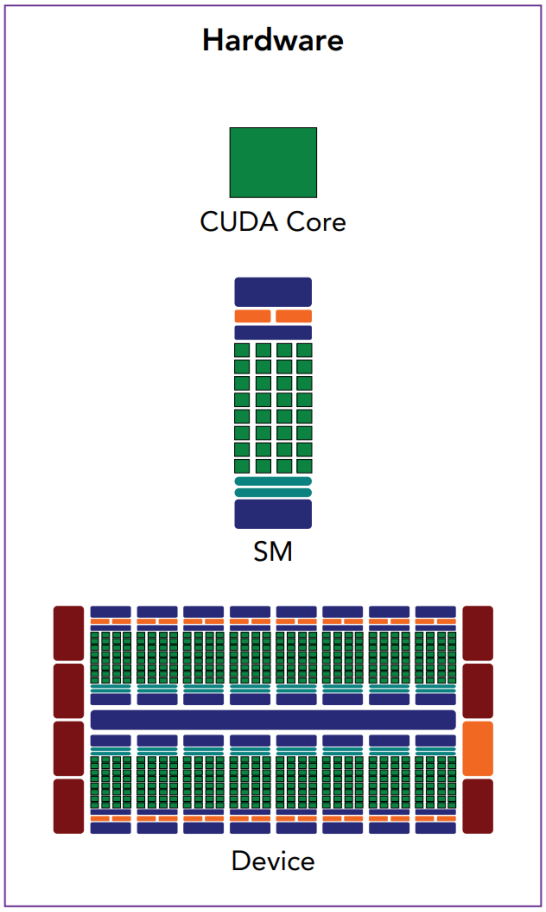
- The SIMT architecture is **similar to the SIMD** (Single Instruction, Multiple Data) architecture.

- **A key difference**:
  - SIMD requires that all vector elements in a vector execute together in a unified synchronous group
  - SIMT allows multiple threads in the same warp to execute independently.

- SIMT includes **three key features** that SIMD does not:
  - Each thread has its own instruction address counter.
  - Each thread has its own register state.
  - Each thread can have an independent execution path.

Logical View and Hardware View of CUDA programming





- A thread block is scheduled on only one SM.
- Once a thread block is scheduled on an SM, it remains there until execution completes.
- An SM can hold more than one thread block at the same time.

Thread Execution

- Shared memory is partitioned among thread blocks resident on the SM and registers are partitioned among threads.
  - Threads in a thread block can cooperate and communicate with each other through these resources.

- While all threads in a thread block run logically in parallel, not all threads can execute physically at the same time.
  - CUDA provides a means to synchronize threads within a thread block.
  - No primitives are provided for inter-block synchronization.

Warp Scheduling

- While warps within a thread block may be scheduled in any order, the **number of active warps** is **limited** by SM resources.

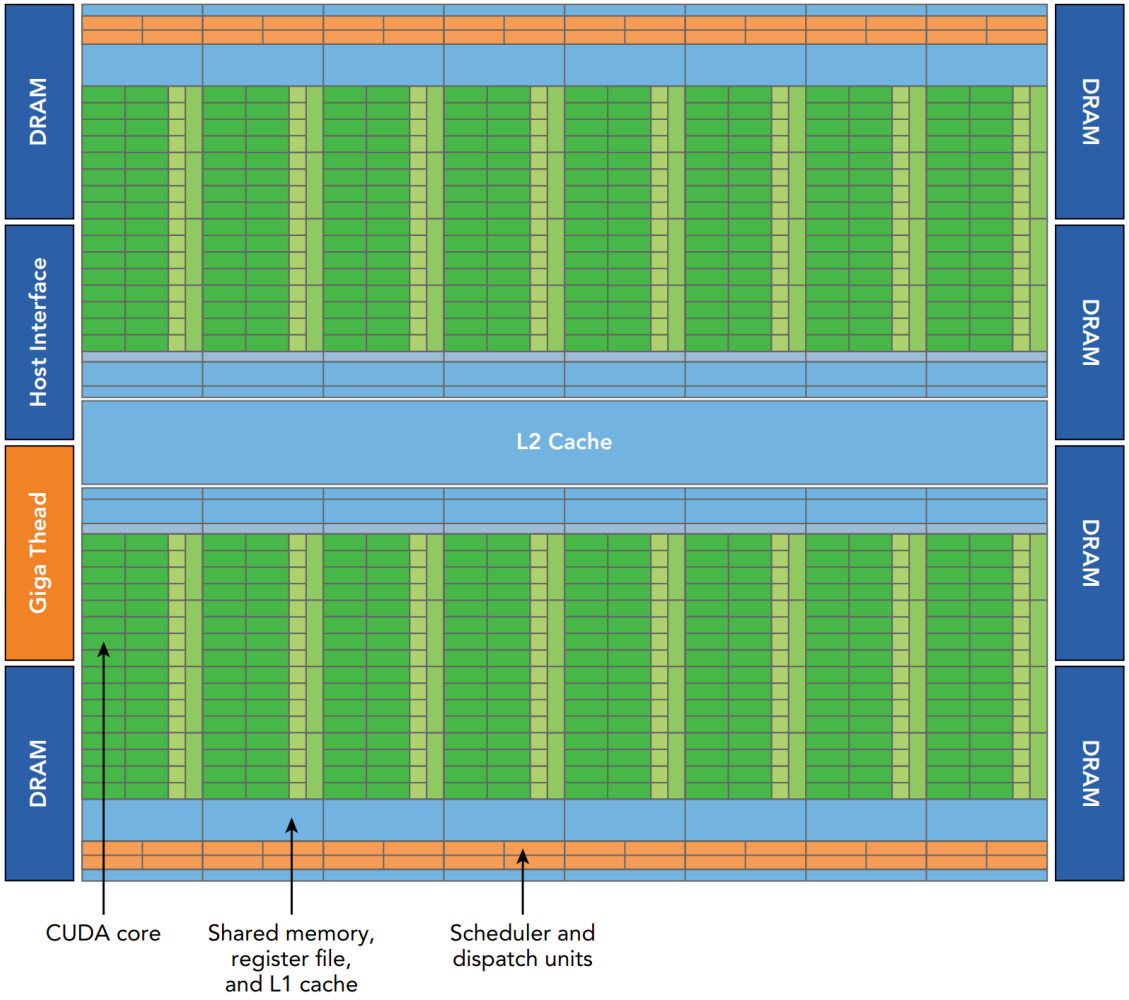
- When a **warp idles** for any reason , the SM is free to **schedule another available warp**
  - from any thread block that is resident on the same SM.

- **Switching** between concurrent warps has **no overhead**
  - Hardware resources are partitioned among all threads and blocks on an SM
  - The state of the newly scheduled warp is already stored on the SM

1.2 Fermi Architecture

Fermi Architecture

- Fermi architecture was the first complete GPU computing architecture for HPC

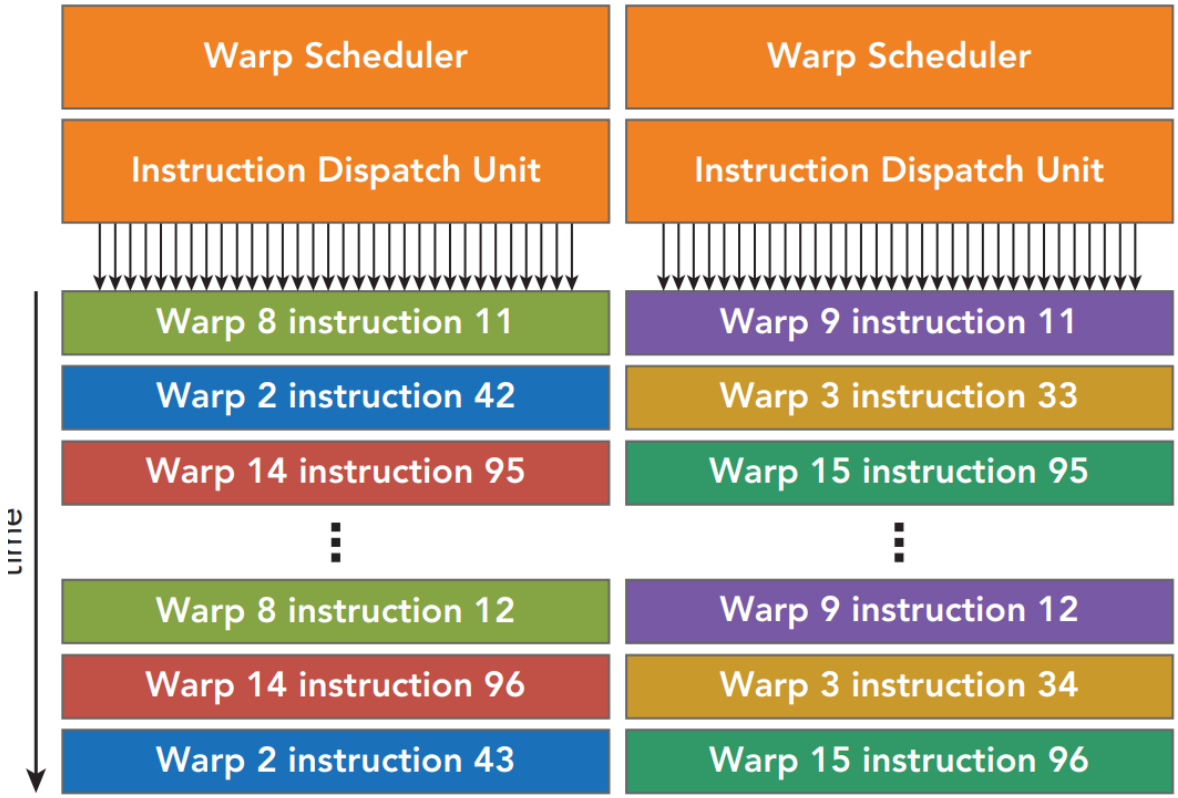


- 512 CUDA cores
- 16 streaming multiprocessors (SM), each with 32 CUDA cores.
- 6 GDDR5 DRAM memory interfaces

GigaThread engine is a global scheduler that distributes thread blocks to the SM warp schedulers.

Fermi Architecture

- Each SM features two warp schedulers and two instruction dispatch units.



- Fermi architecture can simultaneously handle 48 warps per SM for a total of 1,536 threads resident in a single SM at a time

On-chip Configurable Memory

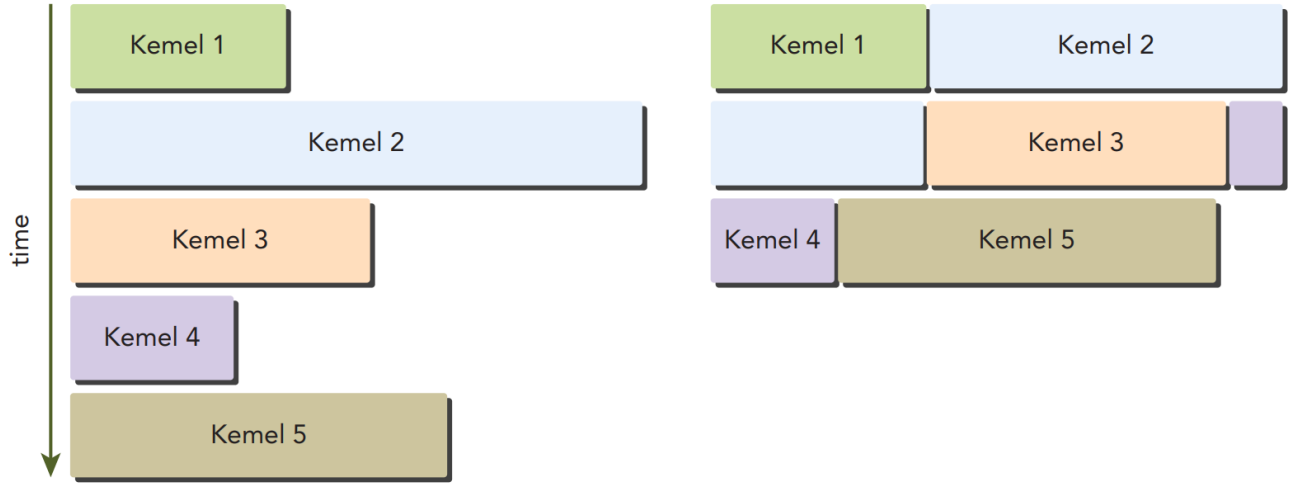
- One key feature of Fermi is the **64 KB on-chip configurable memory**
  - partitioned between **shared memory** and **L1 cache**.

- CUDA provides a **runtime API** that can be used to adjust the amount of shared memory and L1 cache.

- **Modifying on-chip memory configuration** can lead to **performance improvements**
  - depending on the usage of shared memory or cache in a given kernel.

Concurrent Kernel Execution

- Fermi also supports **concurrent kernel execution**
  - multiple kernels launched from the same application context executing on the same GPU at the same time.
  - Concurrent kernel execution allows programs that execute a number of small kernels to fully utilize the GPU.



- Fermi allows up to 16 kernels to be run on the device at the same time.
- Makes the GPU appear more like a MIMD architecture

1.3 Kepler Architecture

Kepler Architecture

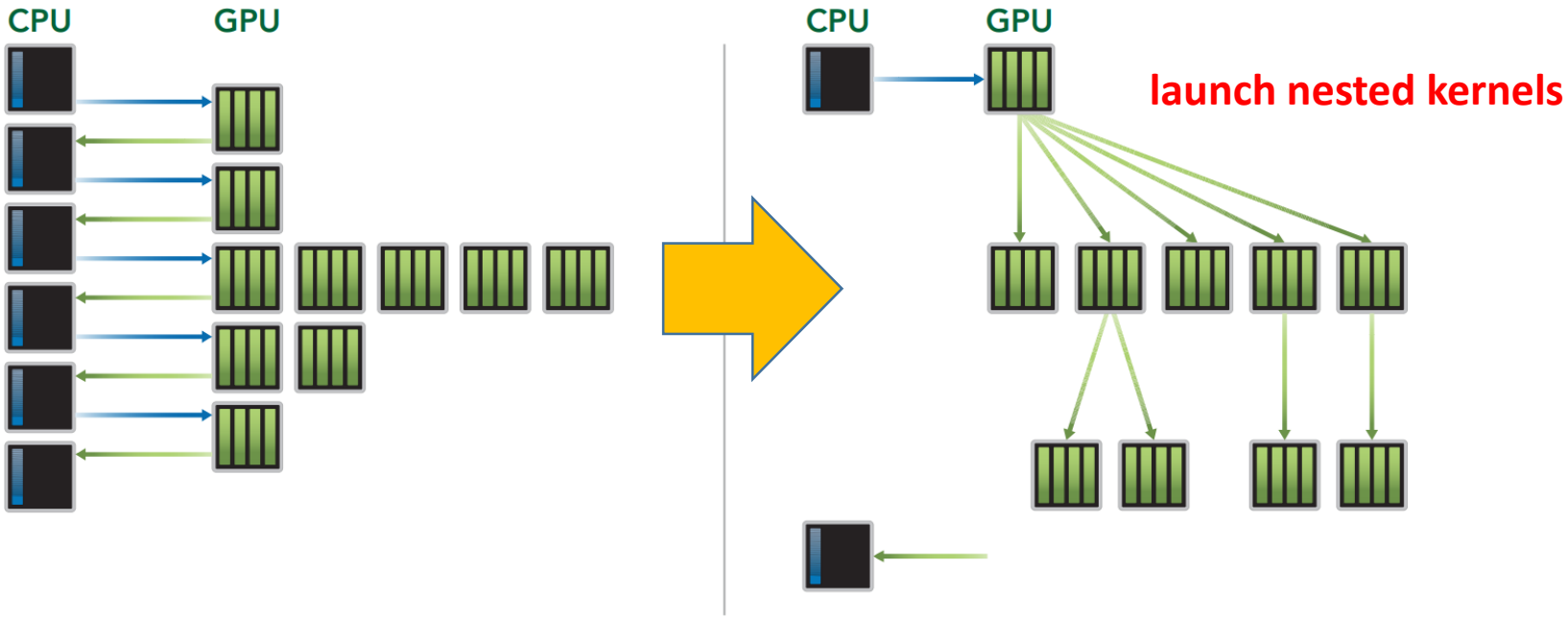
- **Kepler GPU architecture:** released in the fall of 2012
  - 3 key innovations: Enhanced SMs, Dynamic Parallelism, Hyper-Q



- 15 SMs
- Each Kepler SM unit consists of:
  - 192 single-precision CUDA cores
  - 64 double-precision units,
  - 32 special function units (SFU)
  - 32 load/store units (LD/ST)
  - 4 warp schedulers
  - 8 instruction dispatchers
- Schedule 64 warps per SM for a total of 2,048 threads

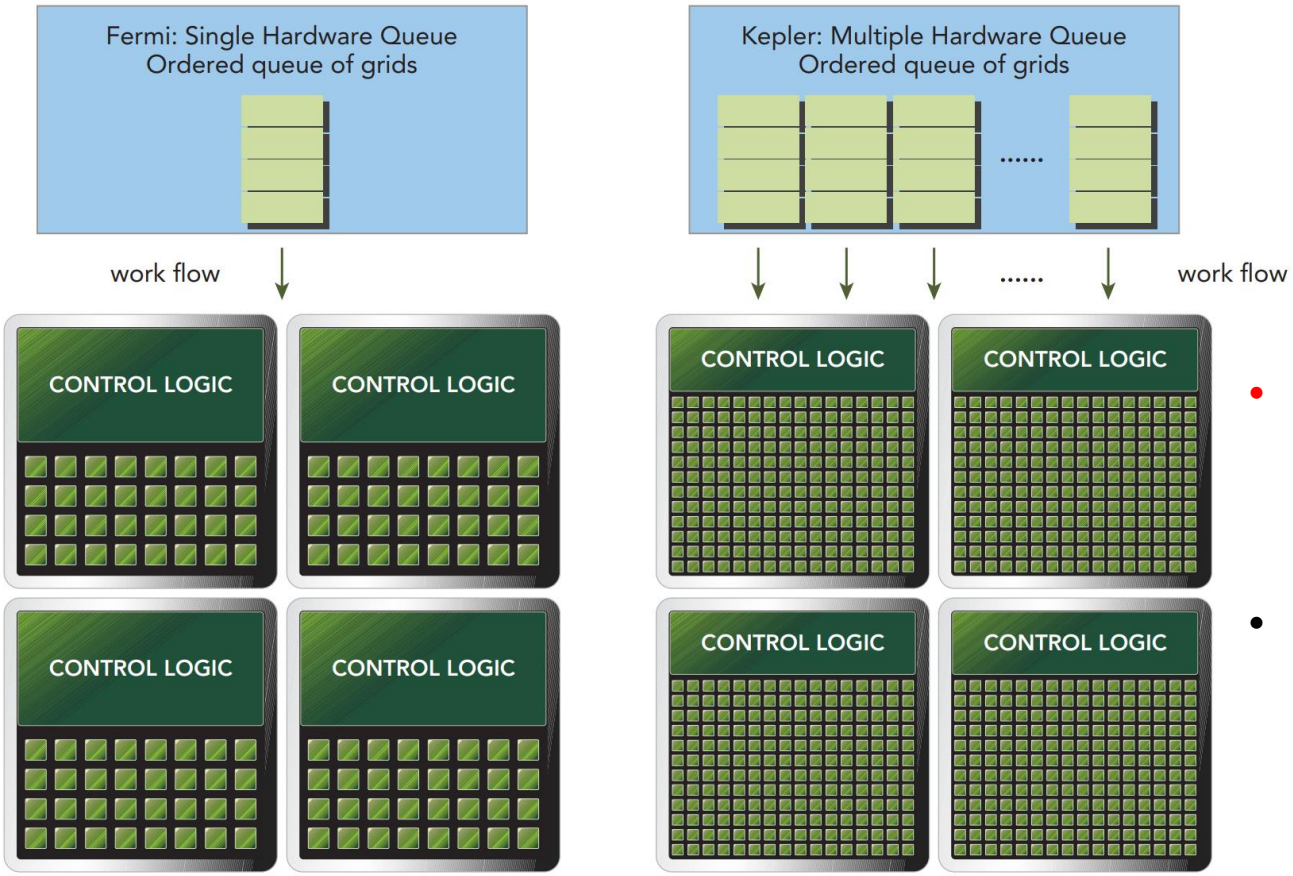
Dynamic Parallelism

- Allows the GPU to **dynamically launch new grids**.
  - Any kernel can launch another kernel and manage any inter-kernel dependencies.
  - Makes it easier to create and optimize recursive and data-dependent execution patterns.



Hyper-Q

- Hyper-Q adds **more simultaneous hardware work queues** between the CPU and GPU
  - enabling CPU to simultaneously run more tasks on GPU



- Avoid a single task to block all other tasks behind it
- Kepler GPUs provide 32 hardware work queues

1.4 Profile-Driven Optimization

Profile-Driven Optimization

- **Profiling** is the act of analyzing program performance by measuring:
  - The space (memory) or time complexity of application code
  - The use of particular instructions
  - The frequency and duration of function calls

- Profiling is a **critical step** in program development, especially for optimizing HPC application code.
  - Often requires a basic understanding of the execution model of a platform.
  - Profiling tools provide deep insight into kernel performance and help you identify bottlenecks in kernels.

Profiling Tools

- CUDA provides two primary **profiling tools:**
  - **`nvvp`**, a standalone visual profiler
  - **`nvprof`**, a command-line profiler

- It is important to choose **appropriate performance metrics** and compare performance.
  - Memory bandwidth
  - Compute resources
  - Instruction and memory latency ……

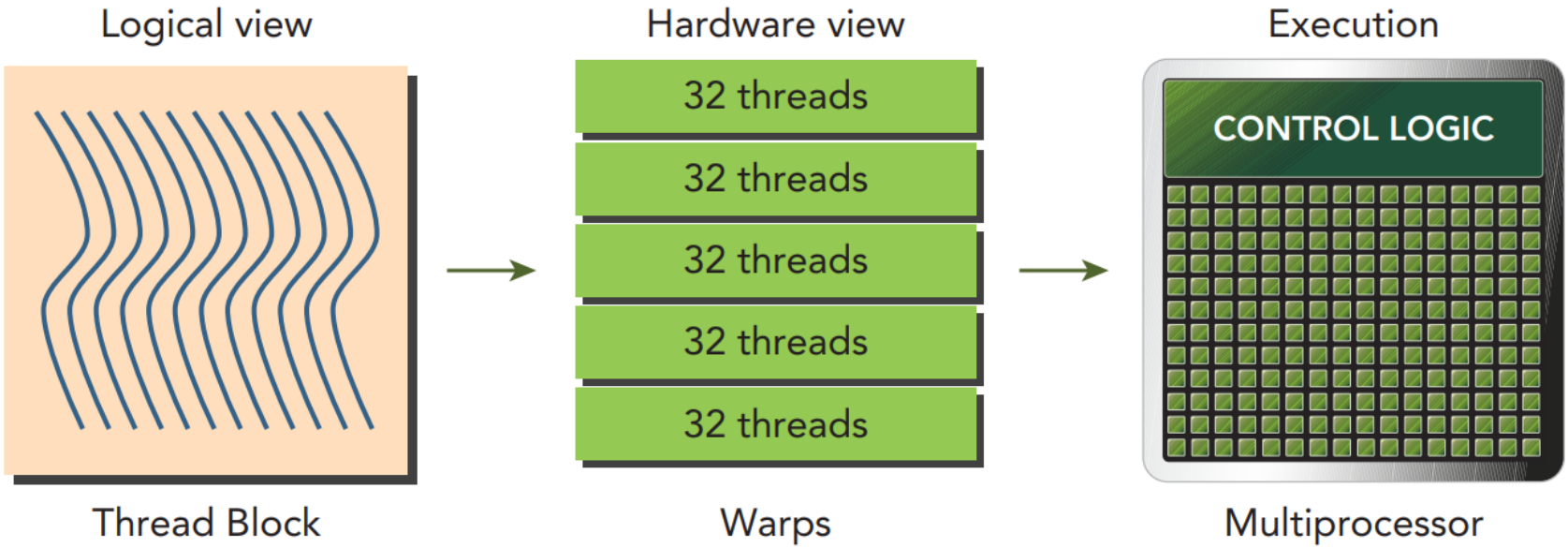
- Even **basic knowledge** of the GPU architecture will enable you to **write much better code** and fully exploit the capability of your device.

2. Understanding the Nature of Warp Execution

2.1 Warps and Thread Blocks

Warps and Thread Blocks

- Warps are the **basic unit of execution** in an SM.
  - A warp consists of 32 consecutive threads
  - When you launch a grid of thread blocks, the thread blocks in the grid are distributed among SMs.
  - Once a thread block is scheduled to an SM, threads in the thread block are further partitioned into warps.



Warps and Thread Blocks

- All threads in a warp are executed in **Single Instruction Multiple Thread (SIMT)** fashion
  - all threads execute the same instruction,
  - each thread carries out that operation on its own private data.

- Thread blocks can be configured to be **one-, two-, or three-dimensional**.

- However, from the **hardware** perspective, all threads are arranged one-dimensionally.

Warps and Thread Blocks

- For a one-dimensional thread **block**, the unique thread ID is stored in the CUDA built-in variable threadIdx.x

- Threads with consecutive values for threadIdx.x are grouped into warps.

- **Example**: a one-dimensional thread block with 128 threads will be organized into 4 warps

```
Warp 0: thread  0, thread  1, thread  2, ... thread 31
Warp 1: thread 32, thread 33, thread 34, ... thread 63
Warp 3: thread 64, thread 65, thread 66, ... thread 95
Warp 4: thread 96, thread 97, thread 98, ... thread 127
```

Warps and Thread Blocks

- The **logical layout of a 2d or 3d thread block** can be converted into its **1-d physical layout**
- For example, given a 2D thread block, a unique identifier for each thread in a block is given by:

threadIdx.y * blockDim.x + threadIdx.x.

- The same calculation for a 3D thread block is as follows::

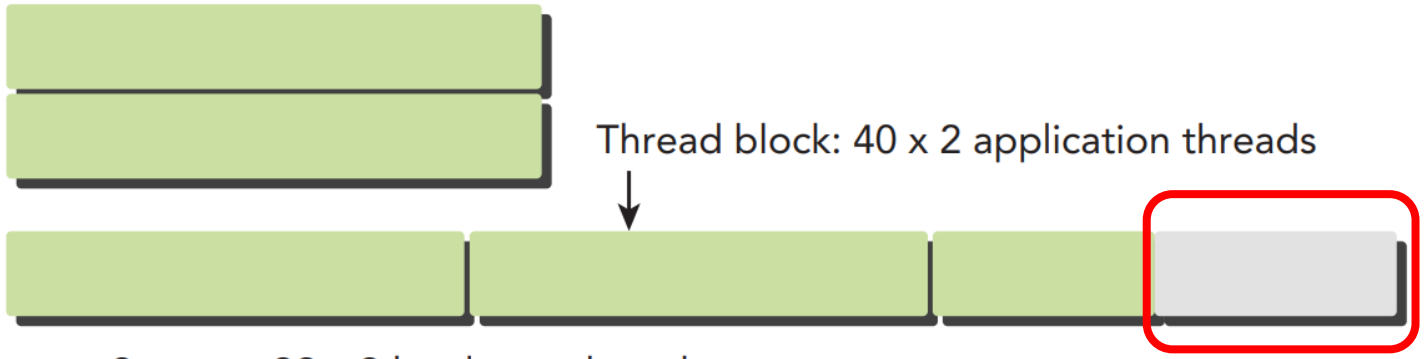
threadIdx.z * blockDim.y * blockDim.x + threadIdx.y * blockDim.x + threadIdx.x

Warps for a Thread Block

- The number of warps for a thread block can be determined as follows:

$$WarpsPerBlock = ceil\left(\frac{ThreadsPerBlock}{warpSize}\right)$$

- If thread block size is not an even multiple of warp size, some threads in the last warp are **left inactive**.



Note that the last half-warp is inactive. Even though these threads are unused they still consume SM resources, such as registers.

Thread Block: Logical View vs. Hardware View

- From the logical perspective, a thread block is a collection of threads organized in a **1D, 2D, or 3D layout**.

- From the hardware perspective, a thread block is a **1D collection of warps**.
  - Threads in a thread block are organized in a 1D layout, and each set of 32 consecutive threads forms a warp.

2.2 Warp Divergence

Warp Divergence

- Threads in the **same warp** executing **different instructions** is referred to as **warp divergence**.
  - All threads in a warp must execute identical instructions on the same cycle
  - This could become a problem if threads in the same warp take different paths through an application.

```
if (cond)
{
...
}
else
{
...
}
```

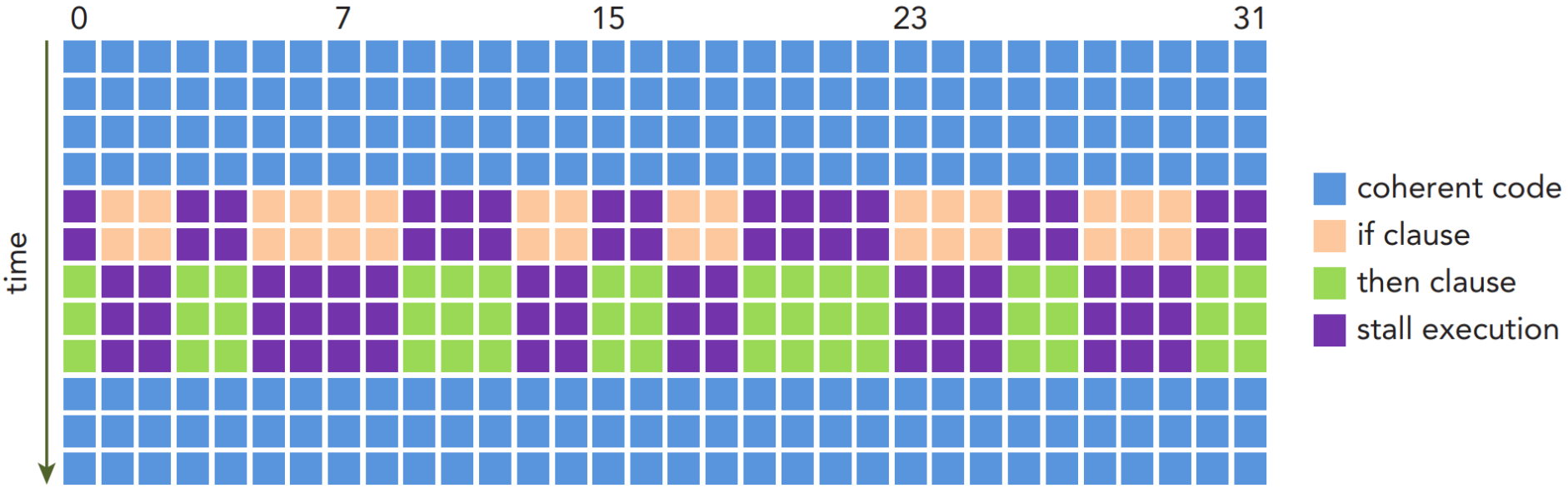
16 threads in a warp

16 threads in the same warp

- Suppose for 16 threads in a warp executing this code, cond is true, but for the other 16 cond is false.
- Then half of the warp will need to execute the instructions in the if block, and the other half will need to execute the instructions in the else block.

Warp Divergence

- If threads of a warp diverge, the warp **serially executes** each branch path, **disabling threads** that do **not take** that path.
  - Cause significantly degraded performance.
  - Branch divergence occurs only within a warp.



All threads within a warp must take both branches of the if…then statement. If the condition is true for a thread, it executes the if clause; otherwise, the thread stalls while waiting for that execution to complete

Warp Divergence

- To obtain the best performance, you should avoid different execution paths within the same warp

- A case with warp divergence

```
__global__ void mathKernel1(float *c) {
  int tid = blockIdx.x * blockDim.x + threadIdx.x;
  float a, b;

  a = b = 0.0f;
  if (tid % 2 == 0) {
    a = 100.0f;
  } else {
    b = 200.0f;
  }
  c[tid] = a + b;
}
```

The condition (tid%2==0) makes even numbered threads take the if clause and odd numbered threads take the else clause.

Avoid Warp Divergence

- If interleave data using a warp approach (instead of a thread approach), you can avoid warp divergence.
  - The condition `(tid/warpSize)%2==0` forces the branch granularity to be a multiple of warp size

```
__global__ void mathKernel2(void) {
  int tid = blockIdx.x * blockDim.x + threadIdx.x;
  float a, b;
  a = b = 0.0f;

  if ((tid / warpSize) % 2 == 0) {
    a = 100.0f;
  } else {
    b = 200.0f;
  }
  c[tid] = a + b;
}
```



simpleDivergence.cu

- The even warps take the if clause, and the odd warps take the else clause.

Branch Efficiency

- **Branch Efficiency** is defined as the ratio of non-divergent branches to total branches

$$Branch\ Efficiency = 100 \times \left(\frac{\#\ Branches - \#\ Divergent\ Branches}{\#\ Branches} \right)$$

- CUDA compiler optimization can replace branch instructions with predicated instructions for short, conditional code segments
  - You can force the CUDA compiler to not optimize your kernel using branch predication

Key Reminders

- Warp divergence occurs when threads within a warp take different code paths.
- Different if-then-else branches are executed serially.
- Try to adjust branch granularity to be a multiple of warp size to avoid warp divergence.
- Different warps can execute different code with no penalty on performance.

2.3 Resource Partitioning

Resource Partitioning

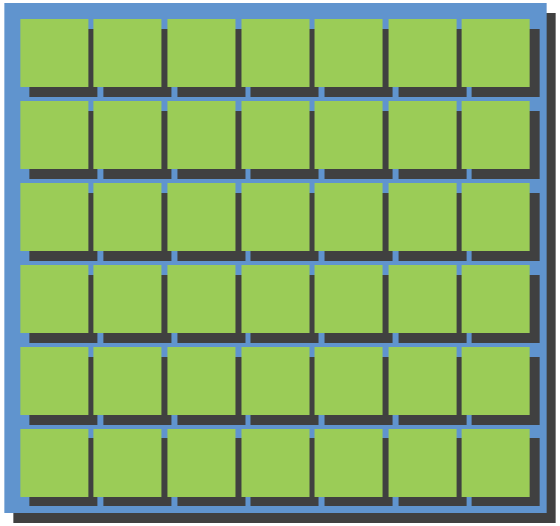
- The local **execution context** of a warp mainly consists of the following resources:
  - Program counters
  - Registers
  - Shared memory

- The execution context of each warp processed by an SM is **maintained on-chip** during the entire lifetime of the warp.
  - Switching from one execution context to another has no cost.

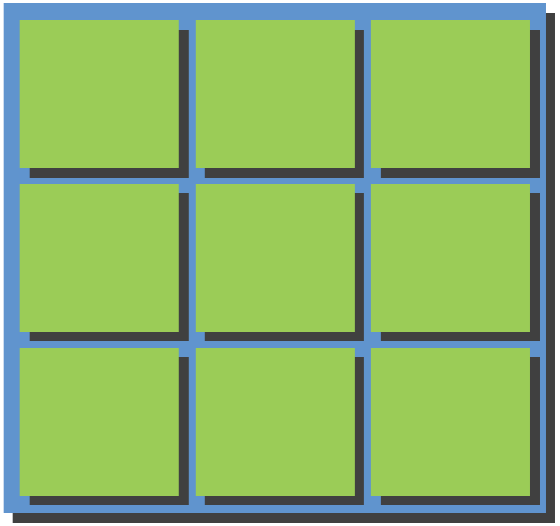
Resource Partitioning

- When each thread consumes more registers, fewer warps can be placed on an SM.
- The number of registers and the amount of shared memory per SM vary for devices of different compute capability.





More threads with fewer registers per thread



Fewer threads with more registers per thread

Resource Limits Pertaining to Compute Capability

TECHNICAL SPECIFICATIONS	COMPUTE CAPABILITY			
	2.0	2.1	3.0	3.5
Maximum number of threads per block	1,024			
Maximum number of concurrent blocks per multiprocessor	8		16	
Maximum number of concurrent warps per multiprocessor	48		64	
Maximum number of concurrent threads per multiprocessor	1,536		2,048	
Number of 32-bit registers per multiprocessor	32 K		64 K	
Maximum number of 32-bit registers per thread	63		255	
Maximum amount of shared memory per multiprocessor	48 K			

Warp Classification

- **Active block:** a thread block with allocated compute resources, such as registers and shared memory.

- **Active warps:** warps in an active block

- Active warps can be further classified into:
  - Selected warp: a warp that is actively executing
  - Eligible warp: an active warp is ready for execution but not currently executing
  - Stalled warp: a warp not ready for execution

Warp Scheduling

- The warp schedulers on an SM select active warps on every cycle and dispatch them to execution units.

- If a warp is not ready for execution, it is a stalled warp.

- A warp is eligible for execution if both conditions is met:
  - 32 CUDA cores are available for execution.
  - All arguments to the current instruction are ready

Example

- **Example: Kepler SM**
  - the number of active warps at any time from launch to completion <= the architecture limit of 64 concurrent warps.
  - The number of selected warps at any cycle is less than or equal to 4.
  - If a warp stalls, the warp scheduler picks up an eligible warp to execute in its place.

- Compute resource partitioning requires **special attention** in CUDA programming
  - The compute resources limit the number of active warps.

2.4 Latency Hiding

Latency Hiding

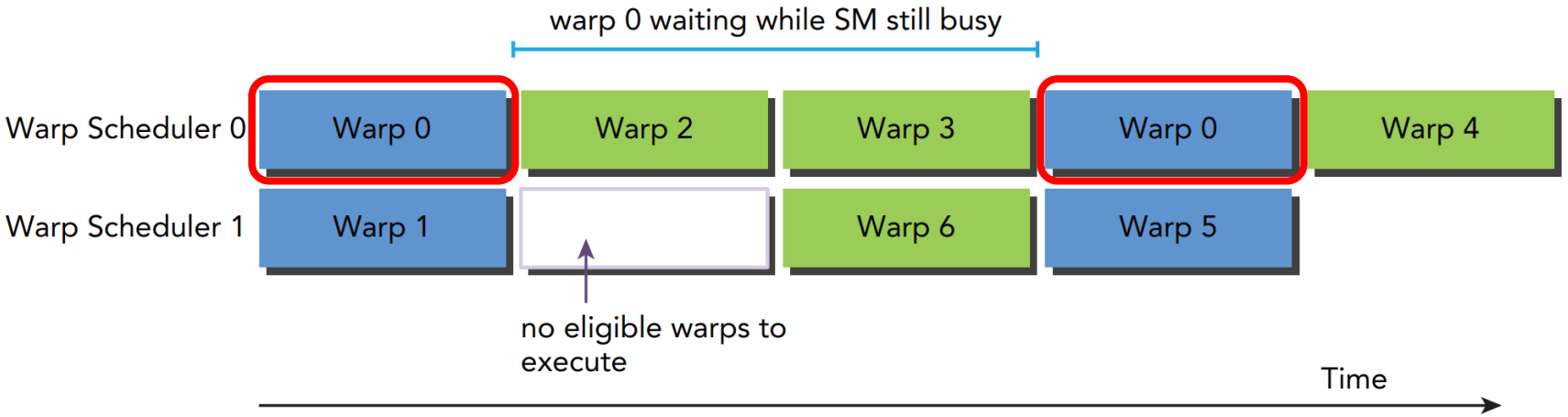
- **Instruction latency:** The number of clock cycles between an instruction being issued and being completed.

- GPU instruction latency is hidden by **computation from other warps**
  - the latency of each instruction can be hidden by issuing other instructions in other resident warps

- Latencies for arithmetic/memory instructions:
  - 10-20 cycles for arithmetic operations
  - 400-800 cycles for global memory accesses

Latency Hiding

- A simple case for an execution pipeline in which warp 0 stalls

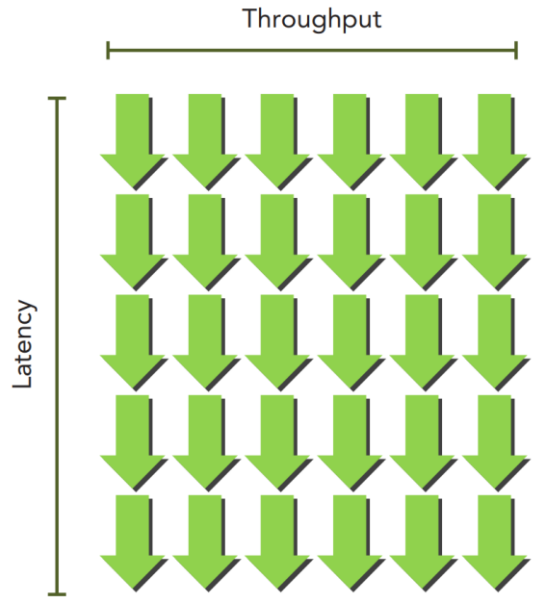


- The warp scheduler picks up other warps to execute and then executes warp 0 when it is eligible again.

Latency Hiding

- **Question:** How to estimate the number of active warps required to hide latency?
- **Answer: Little's Law** can provide a reasonable approximation.

$$Number\ of\ Required\ Warps = Latency \times Throughput$$



- Suppose the average latency for an instruction in your kernel is 5 cycles.
- To keep a throughput of 6 warps executed per cycle, you will need at least 30 warps in-flight

2.5 Occupancy

Occupancy

- **Occupancy** is the ratio of active warps to maximum number of warps, per SM.

$$occupancy = \frac{active\ warps}{maximum\ warps}$$

- You can use cudaGetDeviceProperties to obtain GPU configuration information

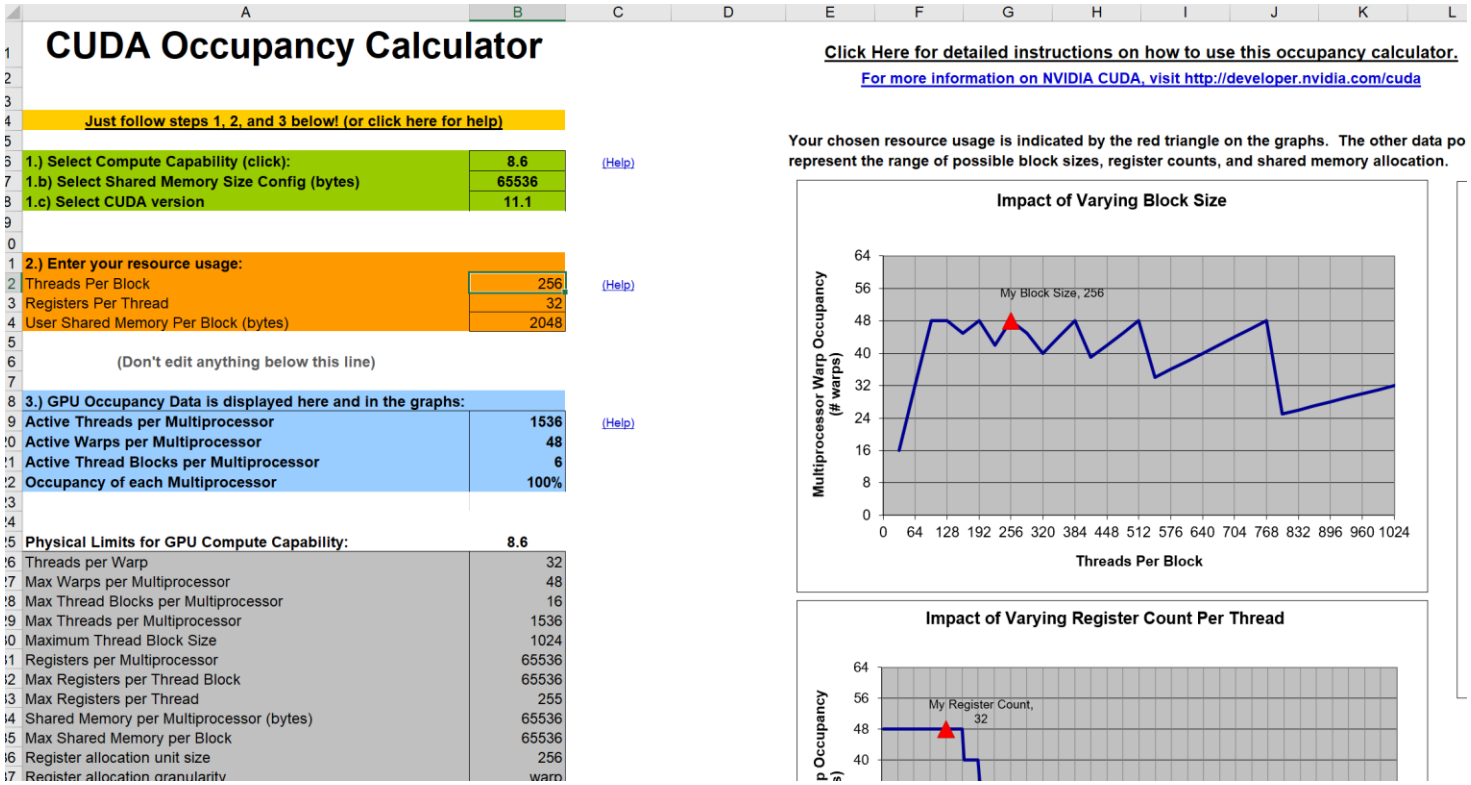
```
Device 0: Tesla M2070
Number of multiprocessors: 14
Total amount of constant memory: 64.00 KB
Total amount of shared memory per block: 48.00 KB
Total number of registers available per block: 32768
Warp size: 32
Maximum number of threads per block: 1024
Maximum number of threads per multiprocessor: 1536
Maximum number of warps per multiprocessor: 48
```



simpleDeviceQuery.cu

CUDA Occupancy Calculator

- The CUDA Toolkit includes a spreadsheet, called the *CUDA Occupancy Calculator*
  - which assists you in selecting grid and block dimensions to maximize occupancy for a kernel.



https://docs.nvidia.com/cuda/cuda-occupancy-calculator

Guidelines for Grid And Block Size

- Keep the number of threads per block a multiple of warp size (32).
- Avoid small block sizes: Start with at least 128 or 256 threads per block.
- Adjust block size up or down according to kernel resource requirements.
- Keep the number of blocks much greater than the number of SMs to expose sufficient parallelism to your device.
- Conduct experiments to discover the best execution configuration and resource usage.

2.6 Synchronization

Synchronization

- In CUDA, synchronization can be performed at two levels:
  - **System-level:** Wait for all work on both host and device to complete.
  - **Block-level:** Wait for all threads in a thread block to reach the same point in execution on the device.

- cudaDeviceSynchronize: used to block the host application until all CUDA operations have completed:

```
cudaDeviceSynchronize(void);
```

Synchronization

- Because warps in a thread block are executed in an undefined order, CUDA provides the ability to synchronize their execution with a block-local barrier.

- You can mark synchronization points in the kernel using:

```
__device__ void __syncthreads(void);
```

When __syncthreads is called, each thread in the same thread block must wait until all other threads in that thread block have reached this synchronization point.

Synchronization

- Threads within a thread block can share data through shared memory and registers.

- When sharing data between threads you need to be careful to **avoid race conditions**.
  - **Race conditions, or hazards**, are unordered accesses by multiple threads to the same memory location.

- **No thread synchronization** among **different blocks**.
  - The only safe way to synchronize across blocks is to use the global synchronization point at the end of every kernel execution
  - By not allowing threads in different blocks to synchronize with each other, GPUs can execute blocks in any order.

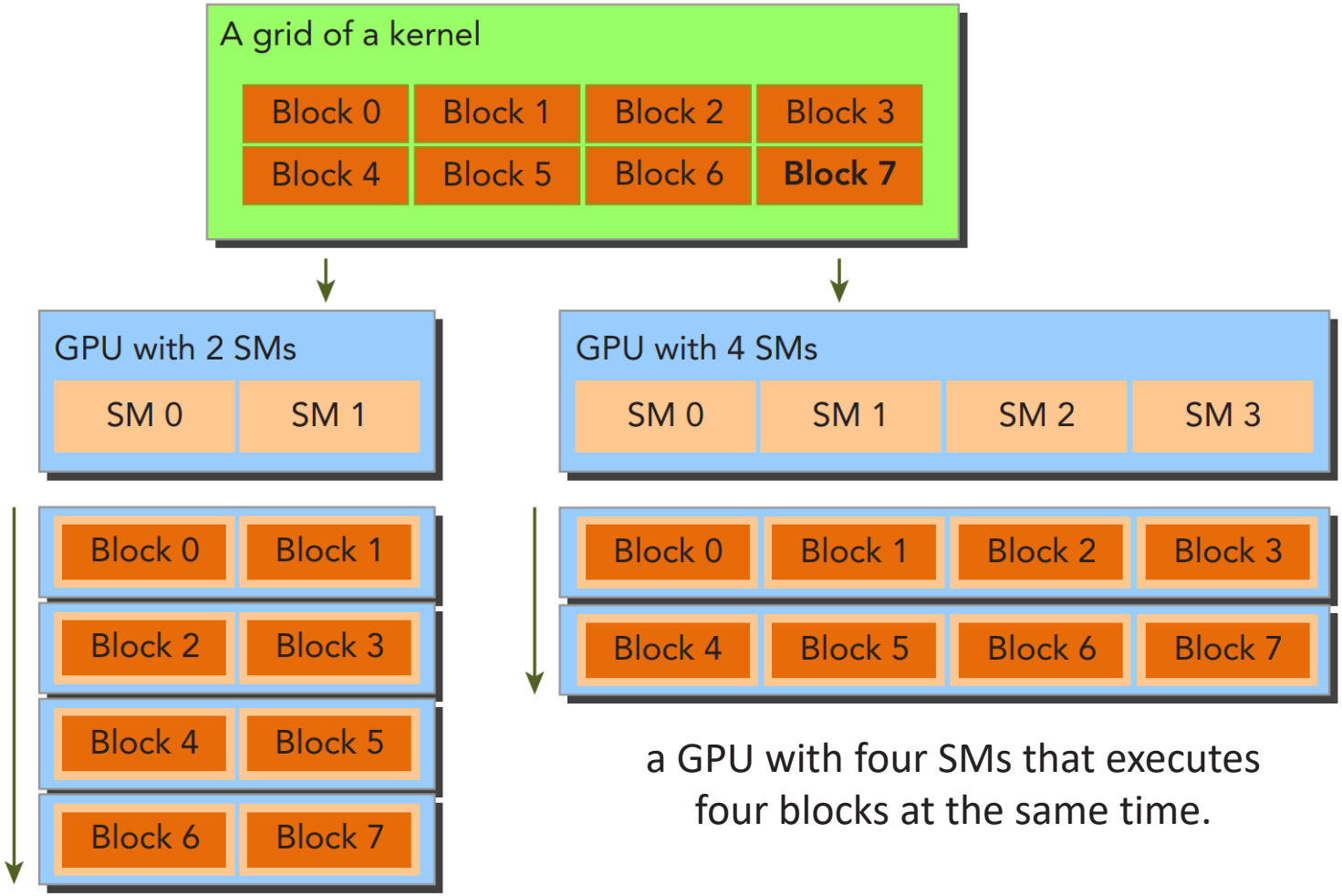
2.7 Scalability

Scalability

- **Scalability** implies that providing additional hardware resources to a parallel application yields speedup relative to the amount of added resources.
  - Scalability implies that performance can be improved with added compute cores.

- **Transparent scalability:** the ability to execute the same application code on a varying number of compute cores.

Scalability



a GPU with two SMs that executes two blocks at the same time

a GPU with four SMs that executes four blocks at the same time.

3. Exposing Parallelism

Exposing Parallelism

- ***grid and block heuristics*** — a must-have skill for CUDA programmers
  - understand why some combinations of grid/block dimensions are better than others

- **Example: 2D matrix summation kernel**

```
__global__ void sumMatrixOnGPU2D(float *A, float *B, float
*C, int NX, int NY) {
  unsigned int ix = blockIdx.x * blockDim.x + threadIdx.x;
  unsigned int iy = blockIdx.y * blockDim.y + threadIdx.y;
  unsigned int idx = iy * NX + ix;

  if (ix < NX && iy < NY) {
    C[idx] = A[idx] + B[idx];
  }
}
```

3.1 Checking Active Warps with nvprof

Checking Active Warps with nvprof

- Test a set of basic thread block configurations: (32,32), (32,16), (16,32), and (16,16).

```
$ ./sumMatrix 32 32
sumMatrixOnGPU2D <<< (512,512),  (32,32) >>> elapsed 60 ms
$ ./sumMatrix 32 16
sumMatrixOnGPU2D <<< (512,1024), (32,16) >>> elapsed 38 ms
$ ./sumMatrix 16 32
sumMatrixOnGPU2D <<< (1024,512), (16,32) >>> elapsed 51 ms
$ ./sumMatrix 16 16
sumMatrixOnGPU2D <<< (1024,1024),(16,16) >>> elapsed 46 ms
```

Fastest!

- **Possible reason**: more thread blocks?



sumMatrix.cu

Checking Active Warps with nvprof

- **Achieved occupancy** of a kernel: the ratio of the average active warps per cycle to the maximum number of warps supported on an SM.

```
$ nvprof --metrics achieved_occupancy ./sumMatrix 32 32
sumMatrixOnGPU2D <<<(512,512),  (32,32)>>> Achieved Occupancy    0.501071
$ nvprof --metrics achieved_occupancy ./sumMatrix 32 16
sumMatrixOnGPU2D <<<(512,1024), (32,16)>>> Achieved Occupancy    0.736900   Fastest!
$ nvprof --metrics achieved_occupancy ./sumMatrix 16 32
sumMatrixOnGPU2D <<<(1024,512), (16,32)>>> Achieved Occupancy    0.766037
$ nvprof --metrics achieved_occupancy ./sumMatrix 16 16
sumMatrixOnGPU2D <<<(1024,1024),(16,16)>>> Achieved Occupancy    0.810691   Highest!
```

The 4-th case has the highest achieved occupancy, but it is not the fastest

- A higher occupancy **does not** always equate to higher performance.

3.2 Checking Memory Operations with nvprof

Checking Memory Ops with nvprof

- 3 memory operations in the sumMatrix kernel (C[idx] = A[idx] + B[idx]):
  - Two memory loads and one memory store.
  - The efficiency of these memory operations can be checked using nvprof.

```
$ nvprof --metrics gld_throughput./sumMatrix 32 32
sumMatrixOnGPU2D <<<(512,512),  (32,32)>>> Global Load Throughput  35.908GB/s
$ nvprof --metrics gld_throughput./sumMatrix 32 16
sumMatrixOnGPU2D <<<(512,1024), (32,16)>>> Global Load Throughput  56.478GB/s
$ nvprof --metrics gld_throughput./sumMatrix 16 32
sumMatrixOnGPU2D <<<(1024,512), (16,32)>>> Global Load Throughput  85.195GB/s
$ nvprof --metrics gld_throughput./sumMatrix 16 16
sumMatrixOnGPU2D <<<(1024,1024),(16,16)>>> Global Load Throughput  94.708GB/s
```

- While the 4th case has the highest load throughput, it is slower than the 2nd case
- a higher load throughput does not always equate to higher performance

Checking Memory Ops with nvprof

- **gld_efficiency**: the ratio of requested global load throughput to required global load throughput.
  - It measures how well the application's load operations use device memory bandwidth.

```
$ nvprof --metrics gld_efficiency ./sumMatrix 32 32
sumMatrixOnGPU2D <<<(512,512),  (32,32)>>> Global Memory Load Efficiency 100.00%
$ nvprof --metrics gld_efficiency ./sumMatrix 32 16
sumMatrixOnGPU2D <<<(512,1024), (32,16)>>> Global Memory Load Efficiency 100.00%
$ nvprof --metrics gld_efficiency ./sumMatrix 16 32
sumMatrixOnGPU2D <<<(1024,512), (16,32)>>> Global Memory Load Efficiency 49.96%
$ nvprof --metrics gld_efficiency ./sumMatrix 16 16
sumMatrixOnGPU2D <<<(1024,1024),(16,16)>>> Global Memory Load Efficiency 49.80%
```

- the load efficiency for the last two cases was half that of the first two cases.
- Explain why the higher load throughput and achieved occupancy of the last two cases did not yield improved performance.

3.3 Exposing More Parallelism

Exposing More Parallelism

- **One conclusion** is that the innermost dimension of a block (block.x) should be a multiple of warp size.
  - Doing so drastically improved load efficiency.

- **Questions**:
  - Is it possible to increase the load throughput further by adjusting block.x?
  - Is it possible to expose more parallelism?

Exposing More Parallelism

```
$ ./sumMatrix 64 2
sumMatrixOnGPU2D <<<(256,8192), (64,2) >>> elapsed 0.033567 sec
$ ./sumMatrix 64 4
sumMatrixOnGPU2D <<<(256,4096), (64,4) >>> elapsed 0.034908 sec
$ ./sumMatrix 64 8
sumMatrixOnGPU2D <<<(256,2048), (64,8) >>> elapsed 0.036651 sec
$ ./sumMatrix 128 2
sumMatrixOnGPU2D <<<(128,8192), (128,2)>>> elapsed 0.032688 sec   Fastest!
$ ./sumMatrix 128 4
sumMatrixOnGPU2D <<<(128,4096), (128,4)>>> elapsed 0.034786 sec
$ ./sumMatrix 128 8
sumMatrixOnGPU2D <<<(128,2048), (128,8)>>> elapsed 0.046157 sec
$ ./sumMatrix 256 2
sumMatrixOnGPU2D <<<(64,8192),  (256,2)>>> elapsed 0.032793 sec
$ ./sumMatrix 256 4
sumMatrixOnGPU2D <<<(64,4096),  (256,4)>>> elapsed 0.038092 sec
$ ./sumMatrix 256 8
sumMatrixOnGPU2D <<<(64,2048),  (256,8)>>> elapsed 0.000173 sec   Error!
Error: sumMatrix.cu:163, code:9, reason: invalid configuration argument
```

- While (64, 2) launches the most thread blocks, it is not the fastest configuration.
- (64, 4) underperforms relative to (128, 2). The size of the innermost dim of a thread block plays a key role in performance.
- All remaining cases have fewer thread blocks than the best case (128, 2).

Exposing More Parallelism

```
$ nvprof --metrics achieved_occupancy ./sumMatrix 64 2
sumMatrixOnGPU2D <<<(256,8192), (64,2) >>>  Achieved Occupancy    0.554556
$ nvprof --metrics achieved_occupancy ./sumMatrix 64 4
sumMatrixOnGPU2D <<<(256,4096), (64,4) >>>  Achieved Occupancy    0.798622
$ nvprof --metrics achieved_occupancy ./sumMatrix 64 8
sumMatrixOnGPU2D <<<(256,2048), (64,8) >>>  Achieved Occupancy    0.753532
$ nvprof --metrics achieved_occupancy ./sumMatrix 128 2
sumMatrixOnGPU2D <<<(128,8192), (128,2)>>>  Achieved Occupancy    0.802598
$ nvprof --metrics achieved_occupancy ./sumMatrix 128 4
sumMatrixOnGPU2D <<<(128,4096), (128,4)>>>  Achieved Occupancy    0.746367
$ nvprof --metrics achieved_occupancy ./sumMatrix 128 8
sumMatrixOnGPU2D <<<(128,2048), (128,8)>>>  Achieved Occupancy    0.573449
$ nvprof --metrics achieved_occupancy ./sumMatrix 256 2
sumMatrixOnGPU2D <<<(64,8192), (256,2) >>>  Achieved Occupancy    0.760901
$ nvprof --metrics achieved_occupancy ./sumMatrix 256 4
sumMatrixOnGPU2D <<<(64,4096), (256,4) >>>  Achieved Occupancy    0.595197
```

Lowest!

Highest!

(64, 2) has the lowest achieved occupancy among all cases, yet it has the most thread blocks.

Metrics and Performance

- In most cases, no single metric can prescribe optimal performance.
- Which metric or event most directly relates to overall performance depends on the nature of the kernel code.
- Seek a good balance among related metrics and events.
- Check the kernel from different angles to find a balance among the related metrics.
- Grid/block heuristics provide a good starting point for performance tuning.

4. Avoiding Branch Divergence

4.1 The Parallel Reduction Problem

The Parallel Reduction Problem

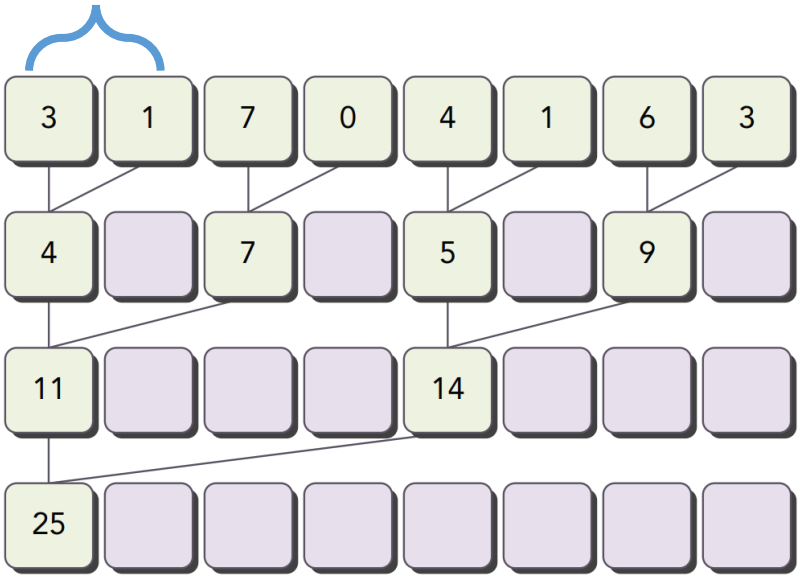
- **Problem**: to calculate the sum of an array of integers with N elements.

```
int sum = 0;
for (int i = 0; i < N; i++)
      sum += array[i];
```

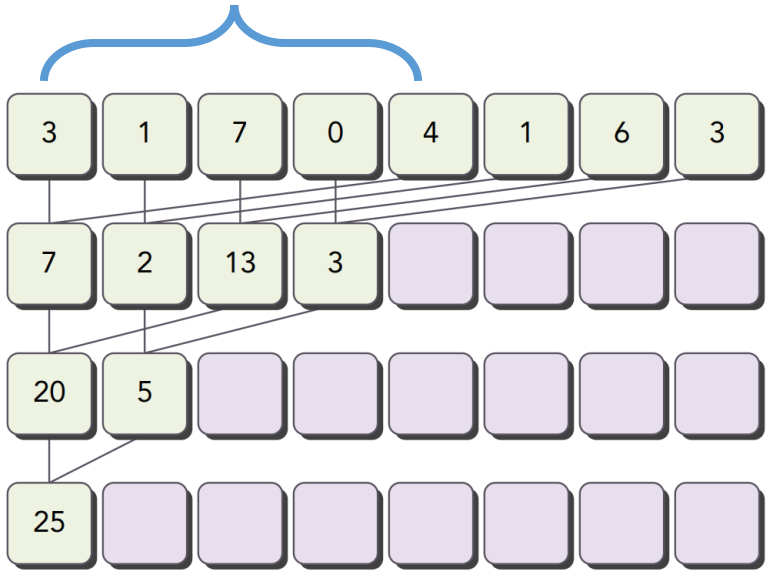
- **Question:** How can you accelerate this sum by executing it in parallel?
- **Parallel addition**
  1. Partition the input vector into smaller chunks.
  2. Have a thread calculate the partial sum for each chunk.
  3. Add the partial results from each chunk into a final sum.

The Parallel Reduction Problem

- Two pairwise parallel sum implementations:
  - **Neighbored pair:** Elements are paired with their immediate neighbor.
  - **Interleaved pair:** Paired elements are separated by a given stride.



Neighbored pair



Interleaved pair

The Parallel Reduction Problem

- **Reduction problem**: general problem of performing a commutative and associative operation (e.g., add, max, avg, min, product) across a vector.

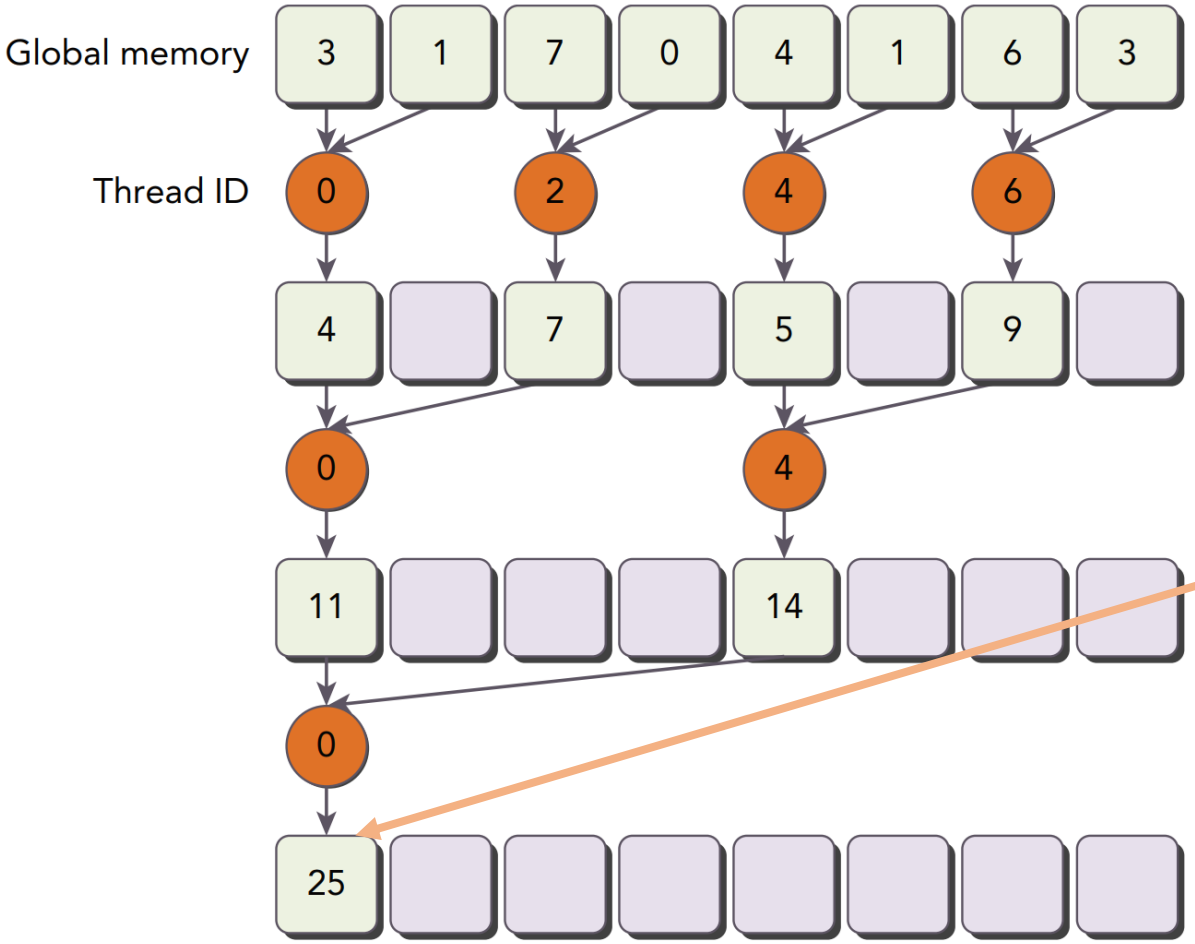
- **Parallel reduction**: the parallel execution of reduction operation.
  - Parallel reduction is one of the most common parallel patterns, and a key operation in many parallel algorithms

Note: While the example above implements addition, any commutative and associative operation could replace addition.

4.2 Divergence in Parallel Reduction

Divergence in Parallel Reduction

- Consider the neighbored pair approach



Two global memory arrays:

(1) one large array for storing the entire array to reduce

(2) one smaller array for holding the partial sums of each thread block.

Divergence in Parallel Reduction

```
__global__ void reduceNeighbored(int *g_idata, int *g_odata, unsigned int n) {
    // set thread ID
    unsigned int tid = threadIdx.x;

    // convert global data pointer to the local pointer of this block
    int *idata = g_idata + blockIdx.x * blockDim.x;

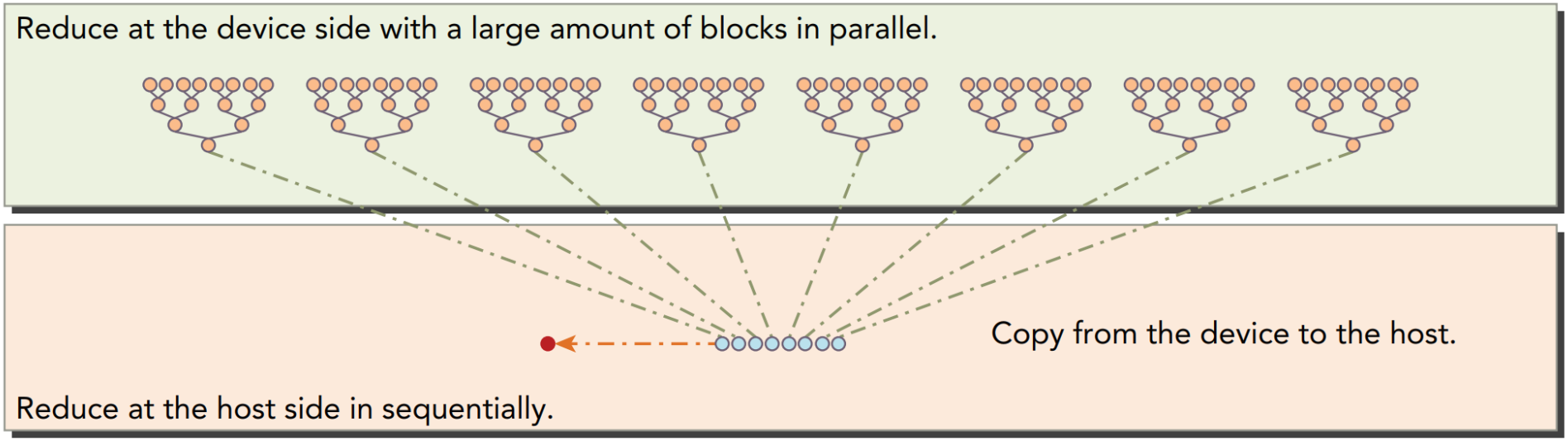
    // boundary check
    if (idx >= n) return;

    // in-place reduction in global memory
    for (int stride = 1; stride < blockDim.x; stride *= 2) {
        if ((tid % (2 * stride)) == 0) {
            idata[tid] += idata[tid + stride];
        }

        // synchronize within block
        __syncthreads();
    }

    // write result for this block to global mem
    if (tid == 0) g_odata[blockIdx.x] = idata[0];
}
```

Divergence in Parallel Reduction



- The distance between two neighbor elements, stride, is initialized to 1 at first.
- After each reduction round, this distance is multiplied by 2.
- After the first round, the even elements of idata will be replaced by partial sums.
- After the second round, every fourth element of idata will be replaced with further partial sums.
- Because there is no synchronization between thread blocks, the partial sum produced by each thread block is copied back to the host and summed sequentially there.



reduceInteger.cu

4.3 Improving Divergence in Parallel Reduction

Warp Divergence

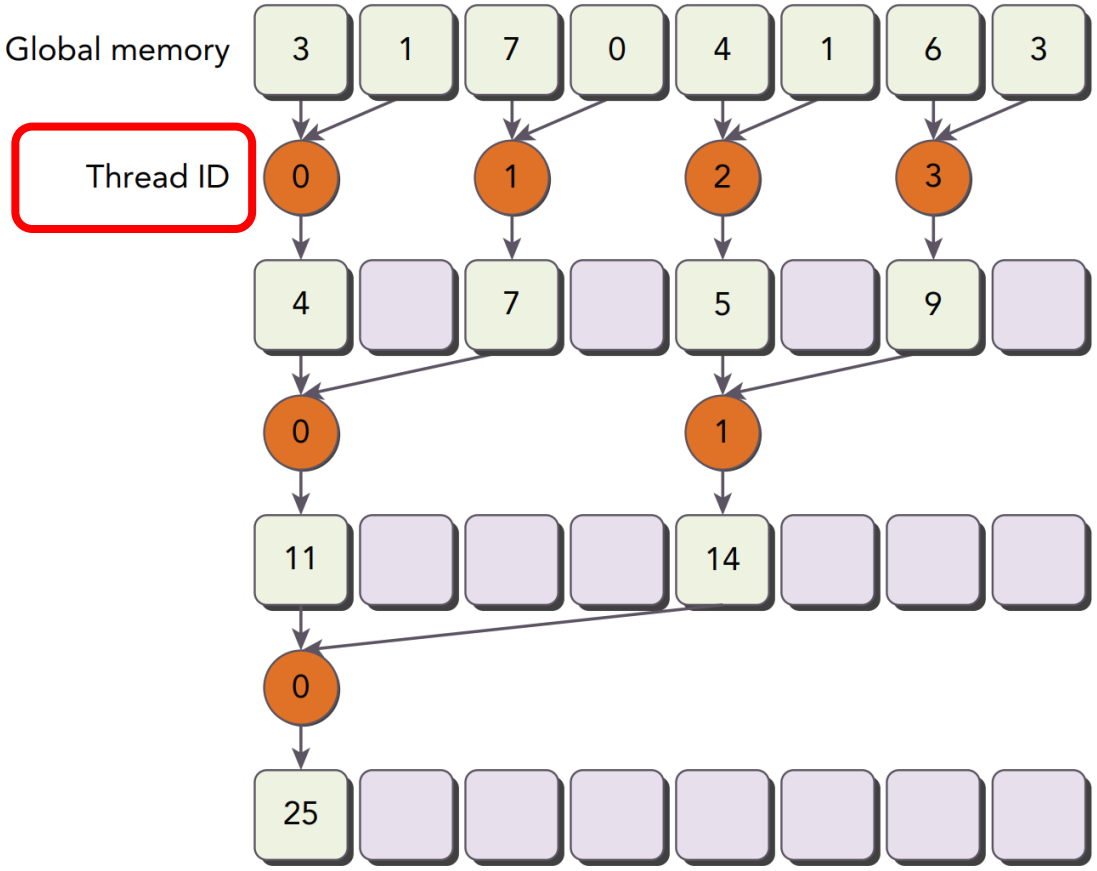
- Examine the kernel reduceNeighbored and the conditional statement:

```
if ((tid % (2 * stride)) == 0)
```

- Only true for even numbered threads, it causes highly divergent warps.
  - In the first iteration of parallel reduction, only even threads execute the body of this conditional statement but all threads must be scheduled.
  - On the second iteration, only one fourth of all threads are active but still all threads must be scheduled.

Improving Divergence

- Warp divergence can be reduced by **rearranging the array index** of each thread to **force neighboring threads** to **perform the addition**.



The store location of partial sums has not changed, but the working threads have been updated.

Improving Divergence

```
__global__ void reduceNeighboredLess (int *g_idata, int *g_odata, unsigned int n) {
    // set thread ID
    unsigned int tid = threadIdx.x;
    unsigned int idx = blockIdx.x * blockDim.x + threadIdx.x;

    // convert global data pointer to the local pointer of this block
    int *idata = g_idata + blockIdx.x*blockDim.x;

    // boundary check
    if(idx >= n) return;

    // in-place reduction in global memory
    for (int stride = 1; stride < blockDim.x; stride *= 2) {
        // convert tid into local array index
        int index = 2 * stride * tid;
        if (index < blockDim.x) {
            idata[index] += idata[index + stride];
        }

        // synchronize within threadblock
        __syncthreads();
    }

    // write result for this block to global mem
    if (tid == 0) g_odata[blockIdx.x] = idata[0];
}
```

sets the array access index for each thread

- With a thread block size of 512 threads, the first 8 warps execute the first round of reduction, and the remaining 8 warps do nothing.
- In the second round, the first 4 warps execute the reduction, and the remaining 12 warps do nothing.
- Therefore, there is no divergence at all.

Improving Divergence

- Testing with reduceNeighboredLess and the earlier kernels produces the following report:

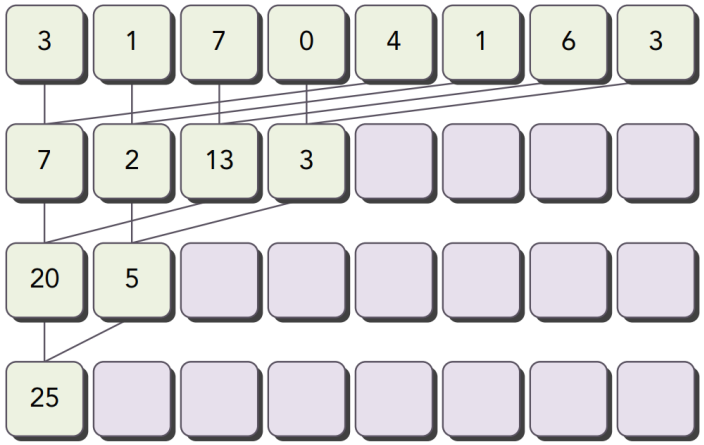
```
$ ./reduceInteger Starting reduction at device 0: Tesla M2070
    vector size 16777216 grid 32768 block 512
cpu reduce      elapsed 0.029138 sec cpu_sum: 2139353471
gpu Neighbored  elapsed 0.011722 sec gpu_sum: 2139353471 <<<grid 32768 block 512>>>
gpu NeighboredL elapsed 0.009321 sec gpu_sum: 2139353471 <<<grid 32768 block 512>>>
```

- The new implementation is 1.26 times faster than the original implementation.

4.4 Reducing with Interleaved Pairs

Reducing with Interleaved Pairs

- The interleaved pair approach **reverses the striding** of elements compared to the neighbored approach
  - The stride is started at half of the thread block size and then reduced by half on each iteration



Each thread adds two elements **separated by the current stride** to produce a partial sum at each round.

- the working threads in interleaved reduction are not changed.
- the load/store locations in global memory for each thread are different.

Reducing with Interleaved Pairs

```
/// Interleaved Pair Implementation with less divergence
__global__ void reduceInterleaved (int *g_idata, int *g_odata, unsigned int n) {
    // set thread ID
    unsigned int tid = threadIdx.x;
    unsigned int idx = blockIdx.x * blockDim.x + threadIdx.x;

    // convert global data pointer to the local pointer of this block
    int *idata = g_idata + blockIdx.x * blockDim.x;

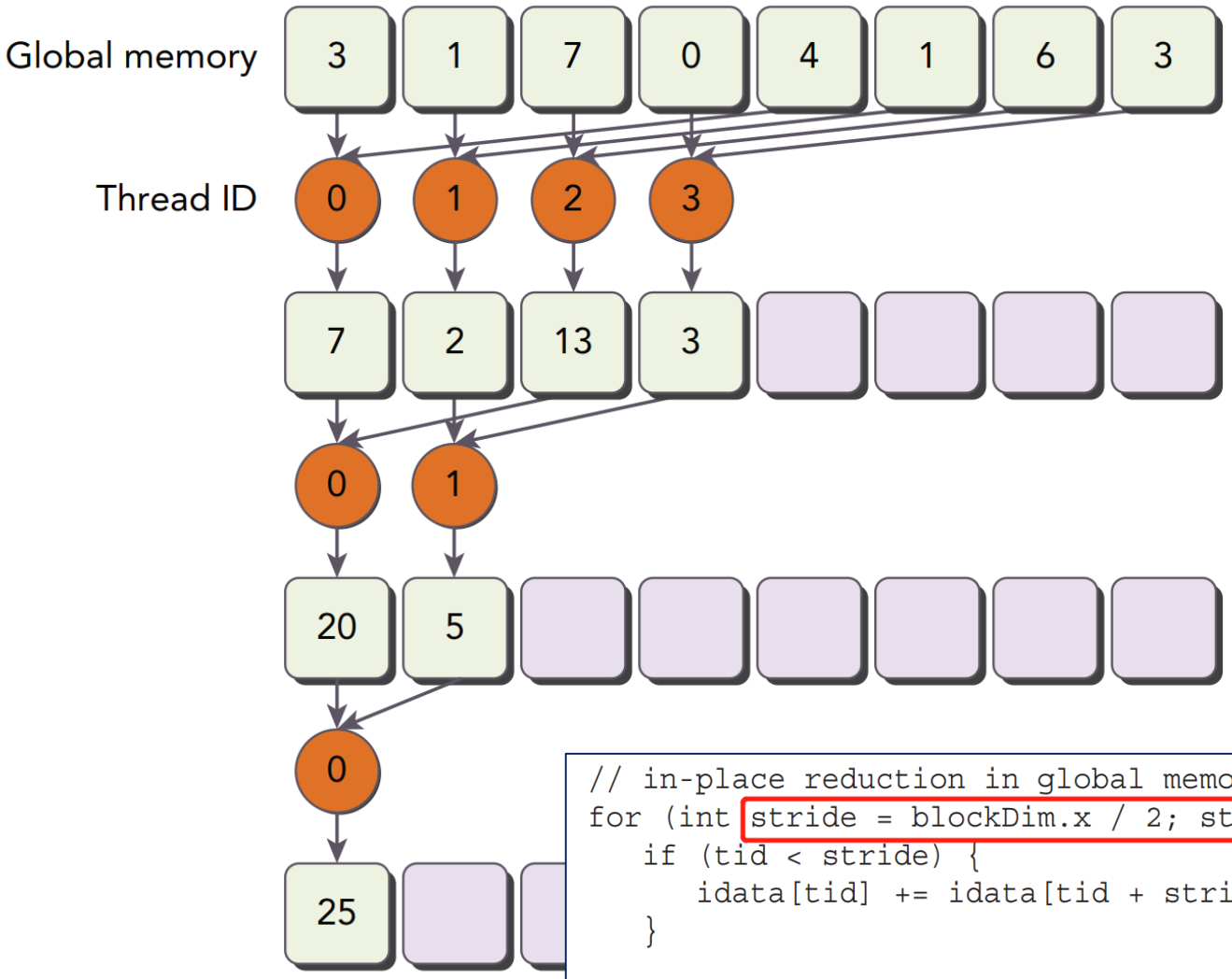
    // boundary check
    if(idx >= n) return;

    // in-place reduction in global memory
    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) {
            idata[tid] += idata[tid + stride];
        }

        __syncthreads();
    }

    // write result for this block to global mem
    if (tid == 0) g_odata[blockIdx.x] = idata[0];
}
```

Reducing with Interleaved Pairs




```
// in-place reduction in global memory
for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
   if (tid < stride) {
      idata[tid] += idata[tid + stride];
   }

   __syncthreads();
}
```

Reducing with Interleaved Pairs

- Testing with reduceInterleaved and the earlier kernels produces the following output:

```
$ ./reduce starting reduction at device 0: Tesla M2070
     with array size 16777216  grid 32768 block 512
cpu reduce      elapsed 0.029138 sec cpu_sum: 2139353471
gpu Warmup      elapsed 0.011745 sec gpu_sum: 2139353471 <<<grid 32768 block 512>>>
gpu Neighbored  elapsed 0.011722 sec gpu_sum: 2139353471 <<<grid 32768 block 512>>>
gpu NeighboredL elapsed 0.009321 sec gpu_sum: 2139353471 <<<grid 32768 block 512>>>
gpu Interleaved elapsed 0.006967 sec gpu_sum: 2139353471 <<<grid 32768 block 512>>>
```

- The interleaved implementation is **1.69 times** faster than the first implementation and **1.34 times faster** than the second implementation.
- This performance improvement is primarily a result of the **global memory load and store patterns** in reduceInterleaved.

5. Unrolling Loops

5.1 Reducing with Unrolling

Loop Unrolling

- **Loop unrolling**: a technique that attempts to optimize loop execution by reducing the frequency of branches and loop maintenance instructions.
  - The loop body is written in code multiple times.
  - The number of loop iterations is reduced or removed entirely

```
for (int i = 0; i < 100; i++) {
   a[i] = b[i] + c[i];
}
```



```
for (int i = 0; i < 100; i += 2) {
   a[i]   = b[i]   + c[i];
   a[i+1] = b[i+1] + c[i+1];

}
```

- **Loop unrolling factor**: the number of copies made of the loop body

Reducing with Unrolling

- A revision to the reduceInterleaved kernel
  - For each thread block, data from two data blocks is summed.

```
__global__ void reduceUnrolling2 (int *g_idata, int *g_odata, unsigned int n) {
   // set thread ID
   unsigned int tid = threadIdx.x;
   unsigned int idx = blockIdx.x * blockDim.x * 2 + threadIdx.x;

   // convert global data pointer to the local pointer of this block
   int *idata = g_idata + blockIdx.x * blockDim.x * 2;

   // unrolling 2 data blocks
   if (idx + blockDim.x < n) g_idata[idx] += g_idata[idx + blockDim.x];
   __syncthreads();

   // in-place reduction in global memory
   for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
      if (tid < stride) {
         idata[tid] += idata[tid + stride];
      }


     // synchronize within threadblock
      __syncthreads();
   }

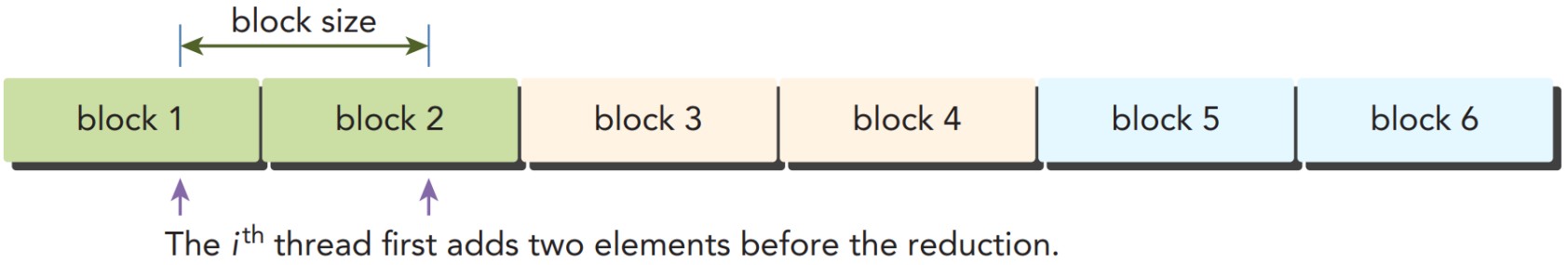
   // write result for this block to global mem
   if (tid == 0) g_odata[blockIdx.x] = idata[0];
}
```

- Each thread is adding an element from the neighboring data block
- Conceptually, you can think of this as an iteration of the reduction loop that reduces across data blocks

Reducing with Unrolling

- The global array index is **adjusted accordingly**
  - because you only need half as many thread blocks to process the same data set.
- The data access for each thread:

```
unsigned int idx = blockIdx.x * blockDim.x * 2 + threadIdx.x;
int *idata = g_idata + blockIdx.x * blockDim.x * 2;
```



Reducing with Unrolling

- The reduceUnrolling2 kernel is now executing **3.42 times faster** than the original implementation

```
gpu Unrolling2 elapsed 0.003430 sec gpu_sum: 2139353471 <<<grid 16384 block 512>>>
gpu Unrolling4 elapsed 0.001829 sec gpu_sum: 2139353471 <<<grid 8192 block 512>>>
gpu Unrolling8 elapsed 0.001422 sec gpu_sum: 2139353471 <<<grid 4096 block 512>>>
```

- More independent memory load/store operations in a single thread yield better performance
  - As memory latency can be better hidden.

```
Unrolling2 Device Memory Read Throughput  26.295GB/s
Unrolling4 Device Memory Read Throughput  49.546GB/s
Unrolling8 Device Memory Read Throughput  62.764GB/s
```

5.2 Reducing with Unrolled Warps

Reducing with Unrolled Warps

- Consider that there are 32 or fewer threads left (that is, a single warp).
  - Because warp execution is SIMT, there is implicit intra-warp synchronization after each instruction.
  - The last 6 iterations of the reduction loop can be unrolled:

```
if (tid < 32) {
    volatile int *vmem = idata;
    vmem[tid] += vmem[tid + 32];
    vmem[tid] += vmem[tid + 16];
    vmem[tid] += vmem[tid + 8];
    vmem[tid] += vmem[tid + 4];
    vmem[tid] += vmem[tid + 2];
    vmem[tid] += vmem[tid + 1];
}
```

- volatile tells the compiler that it must store vmem[tid] back to global memory with every assignment.
- Any reference to volatile variables forces a read or write directly to memory, and not simply to cache or a register.

Reducing with Unrolled Warps

```
__global__ void reduceUnrollWarps8 (int *g_idata, int *g_odata, unsigned int n) {
   // set thread ID
   unsigned int tid = threadIdx.x;
   unsigned int idx = blockIdx.x*blockDim.x*8 + threadIdx.x;

   // convert global data pointer to the local pointer of this block
   int *idata = g_idata + blockIdx.x*blockDim.x*8;

   // unrolling 8
   if (idx + 7*blockDim.x < n) {
      int a1 = g_idata[idx];
      int a2 = g_idata[idx+blockDim.x];
      int a3 = g_idata[idx+2*blockDim.x];
      int a4 = g_idata[idx+3*blockDim.x];
      int b1 = g_idata[idx+4*blockDim.x];
      int b2 = g_idata[idx+5*blockDim.x];
      int b3 = g_idata[idx+6*blockDim.x];
      int b4 = g_idata[idx+7*blockDim.x];
      g_idata[idx] = a1+a2+a3+a4+b1+b2+b3+b4;
   }
   __syncthreads();
```

Reducing with Unrolled Warps

```
// in-place reduction in global memory
for (int stride = blockDim.x / 2; stride > 32; stride >>= 1) {
   if (tid < stride) {
      idata[tid] += idata[tid + stride];
   }

  // synchronize within threadblock
   __syncthreads();
}

// unrolling warp
if (tid < 32) {
   volatile int *vmem = idata;
   vmem[tid] += vmem[tid + 32];
   vmem[tid] += vmem[tid + 16];
   vmem[tid] += vmem[tid +  8];
   vmem[tid] += vmem[tid +  4];
   vmem[tid] += vmem[tid +  2];
   vmem[tid] += vmem[tid +  1];
}

// write result for this block to global mem
if (tid == 0) g_odata[blockIdx.x] = idata[0];
}
```

The execution time for this kernel is

- **1.05 times faster** than reduceUnrolling8
- **8.65 times faster** than the original kernel reduceNeighbored

5.3 Reducing with Complete Unrolling

Reducing with Complete Unrolling

- If you know the number of iterations in a loop at compile-time, you can completely unroll it.
  - Suppose the maximum # of threads per block to be 1024

```
__global__ void reduceCompleteUnrollWarps8 (int *g_idata, int *g_odata,
      unsigned int n) {
   // set thread ID
   unsigned int tid = threadIdx.x;
   unsigned int idx = blockIdx.x * blockDim.x * 8 + threadIdx.x;

   // convert global data pointer to the local pointer of this block
   int *idata = g_idata + blockIdx.x * blockDim.x * 8;

   // unrolling 8
   if (idx + 7*blockDim.x < n) {
      int a1 = g_idata[idx];
      int a2 = g_idata[idx + blockDim.x];
      int a3 = g_idata[idx + 2 * blockDim.x];
      int a4 = g_idata[idx + 3 * blockDim.x];
      int b1 = g_idata[idx + 4 * blockDim.x];
      int b2 = g_idata[idx + 5 * blockDim.x];
      int b3 = g_idata[idx + 6 * blockDim.x];
      int b4 = g_idata[idx + 7 * blockDim.x];
      g_idata[idx] = a1 + a2 + a3 + a4 + b1 + b2 + b3 + b4;
   }
   __syncthreads();
```

Reducing with Complete Unrolling

```
// in-place reduction and complete unroll
if (blockDim.x>=1024 && tid < 512) idata[tid] += idata[tid + 512];
__syncthreads();

if (blockDim.x>=512 && tid < 256) idata[tid] += idata[tid + 256];
__syncthreads();

if (blockDim.x>=256 && tid < 128) idata[tid] += idata[tid + 128];
__syncthreads();

if (blockDim.x>=128 && tid < 64) idata[tid] += idata[tid + 64];
__syncthreads();
// unrolling warp
if (tid < 32) {
   volatile int *vsmem = idata;
   vsmem[tid] += vsmem[tid + 32];
   vsmem[tid] += vsmem[tid + 16];
   vsmem[tid] += vsmem[tid +  8];
   vsmem[tid] += vsmem[tid +  4];
   vsmem[tid] += vsmem[tid +  2];
   vsmem[tid] += vsmem[tid +  1];
}
// write result for this block to global mem
if (tid == 0) g_odata[blockIdx.x] = idata[0];
}
```

- **1.06** times faster than reduceUnrollWarps8
- **9.16** times faster than the original implementation.

5.4 Reducing with Template Functions

Reducing with Template Functions

- While it is possible to manually unroll loops, using template functions can further reduce branch overhead.
  - CUDA supports template parameters on device functions.

Specify the block size as a parameter of the template function

The only difference

```
template <unsigned int iBlockSize>
__global__ void reduceCompleteUnroll(int *g_idata, int *g_odata, unsigned int n) {
    // set thread ID
    unsigned int tid = threadIdx.x;
    unsigned int idx = blockIdx.x * blockDim.x * 8 + threadIdx.x;

    // convert global data pointer to the local pointer of this block
    int *idata = g_idata + blockIdx.x * blockDim.x * 8;

    // unrolling 8
    if (idx + 7*blockDim.x < n) {
        int a1 = g_idata[idx];
        int a2 = g_idata[idx + blockDim.x];
```

Reducing with Template Functions

- IF statements that check the block size will be evaluated at compile time and removed if the condition is not true
  - resulting in a very efficient inner loop.

- For example, if this kernel were invoked with a thread block size of 256

```
iBlockSize>=1024 && tid < 512
```

- The statement would always be false.
- The compiler would automatically remove it from the executed kernel

Reduction Kernel Performance

KERNEL	TIME (S)	STEP SPEEDUP	CUMULATIVE SPEEDUP
Neighbored (divergence)	0.011722		
Neighbored (no divergence)	0.009321	1.26	1.26
Interleaved	0.006967	1.34	1.68
Unroll 8 blocks	0.001422	4.90	8.24
Unroll 8 blocks + last warp	0.001355	1.05	8.65
Unroll 8 blocks + loop + last warp	0.001280	1.06	9.16
Templatized kernel	0.001253	1.02	9.35

Note that the largest relative performance gain is achieved by the **reduceUnrolling8** kernel, in which each thread handles **8 data blocks** before starting the reduce.

Load/Store Efficiency

KERNEL	TIME (S)	LOAD EFFICIENCY	STORE EFFICIENCY
Neighbored (divergence)	0.011722	16.73%	25.00%
Neighbored (no divergence)	0.009321	16.75%	25.00%
Interleaved	0.006967	77.94%	95.52%
Unroll 8 blocks	0.001422	94.68%	97.71%
Unroll 8 blocks + last warp	0.001355	98.99%	99.40%
Unroll 8 blocks + loop + last warp	0.001280	98.99%	99.40%
Templatized the last kernel	0.001253	98.99%	99.40%

6. Dynamic Parallelism

6.1 Nested Execution

Dynamic Parallelism

- Previously, all kernels have been invoked from the host thread.
  - The GPU workload is completely under the control of the CPU.

- **CUDA Dynamic Parallelism** allows new GPU kernels to be created and synchronized directly on the GPU.
  - Enable a more hierarchical approach where concurrency can be expressed in multiple levels in a GPU kernel.
  - Make your recursive algorithm more transparent and easier to understand.

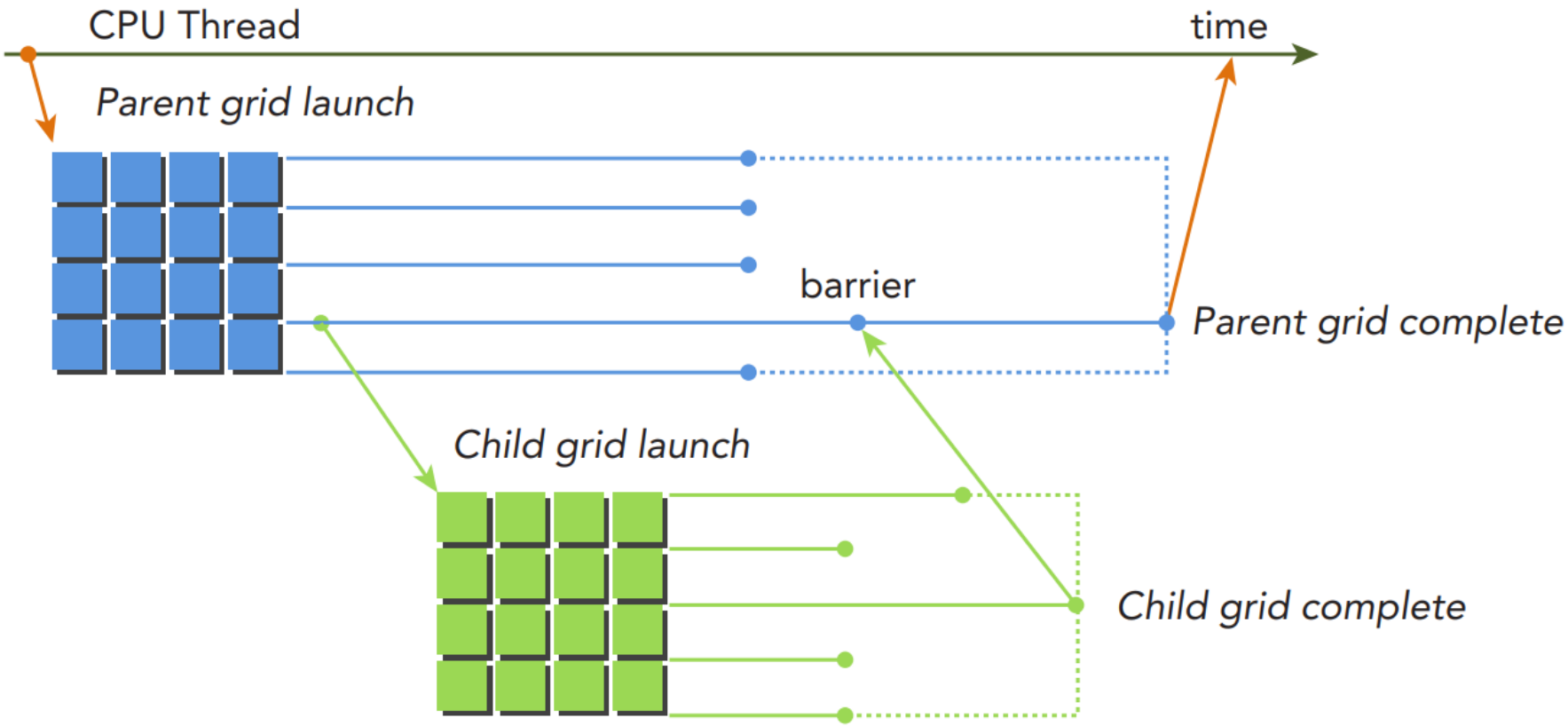
Dynamic Parallelism

- In dynamic parallelism, kernel executions are classified into two types: **parent and child**.
  - A parent thread, parent thread block, or parent grid has launched a new grid, the child grid.

- A child grid must complete before the parent thread, parent thread block, or parent grids are considered complete.

- A parent is not considered complete until all of its child grids have completed.

Nested Execution

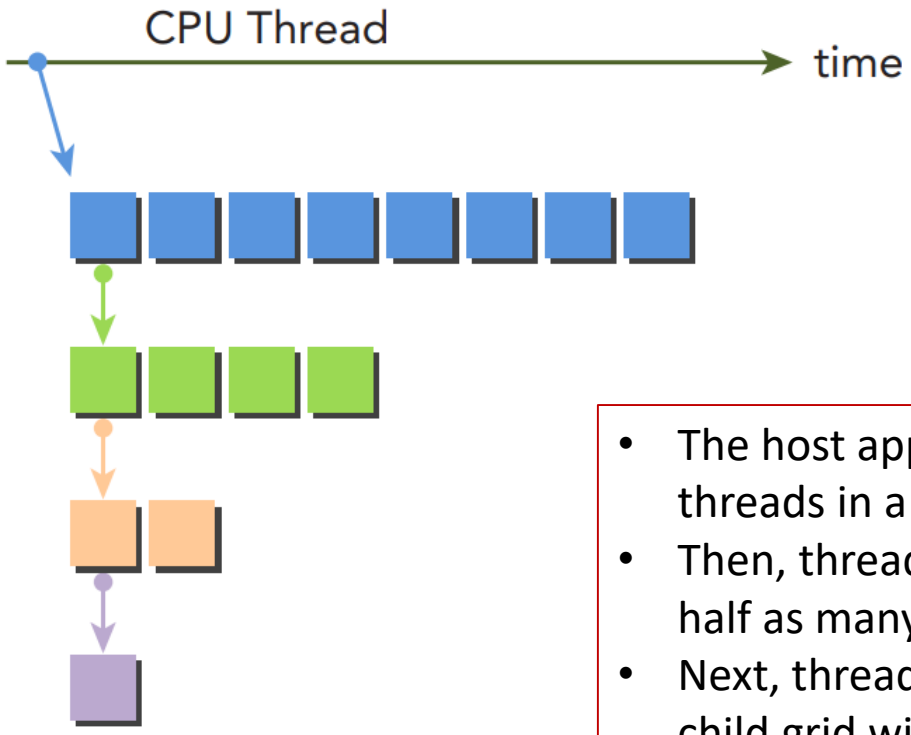


- The parent grid is configured and launched by the host thread, and the child grid is configured and launched by the parent grid.
- A barrier is set in the parent thread to explicitly synchronize with its child grid.

6.2 Nested Hello World on the GPU

Nested Hello World on the GPU

- A nested, recursive execution using dynamic parallelism.



- The host application invokes the parent grid with 8 threads in a single thread block.
- Then, thread 0 in that grid invokes a child grid with half as many threads.
- Next, thread 0 in the first child grid invokes a new child grid with half as many threads again,
- so on until only one thread is left in the final nesting.

Nested Hello World on the GPU

- Execution of the kernel by every thread starts with printing "Hello World".
  - Then, each thread checks if it should terminate.
  - If thread count at this nested layer is greater than one, thread 0 recursively invokes a child grid with half as many threads

```
__global__ void nestedHelloWorld(int const iSize,int iDepth) {
   int tid = threadIdx.x;
   printf("Recursion=%d: Hello World from thread %d"
        "block %d\n",iDepth,tid,blockIdx.x);

   // condition to stop recursive execution
   if (iSize == 1) return;

   // reduce block size to half
   int nthreads = iSize>>1;

   // thread 0 launches child grid recursively
   if(tid == 0 && nthreads > 0) {
      nestedHelloWorld<<<1, nthreads>>>(nthreads,++iDepth);
      printf("-------> nested execution depth: %d\n",iDepth);
   }
}
```

Nested Hello World on the GPU

- $ nvcc -arch=sm_35 -rdc=true nestedHelloWorld.cu -o nestedHelloWorld -lcudadevrt

```
./nestedHelloWorld Execution Configuration: grid 1 block 8
Recursion=0: Hello World from thread 0 block 0
Recursion=0: Hello World from thread 1 block 0
Recursion=0: Hello World from thread 2 block 0
Recursion=0: Hello World from thread 3 block 0
Recursion=0: Hello World from thread 4 block 0
Recursion=0: Hello World from thread 5 block 0
Recursion=0: Hello World from thread 6 block 0
Recursion=0: Hello World from thread 7 block 0
-------> nested execution depth: 1
Recursion=1: Hello World from thread 0 block 0
Recursion=1: Hello World from thread 1 block 0
Recursion=1: Hello World from thread 2 block 0
Recursion=1: Hello World from thread 3 block 0
-------> nested execution depth: 2
Recursion=2: Hello World from thread 0 block 0
Recursion=2: Hello World from thread 1 block 0
-------> nested execution depth: 3
Recursion=3: Hello World from thread 0 block 0
```



nestedHelloWorld.cu

Restrictions On Dynamic Parallelism

- Dynamic Parallelism is only supported by devices of compute capability 3.5 and higher.

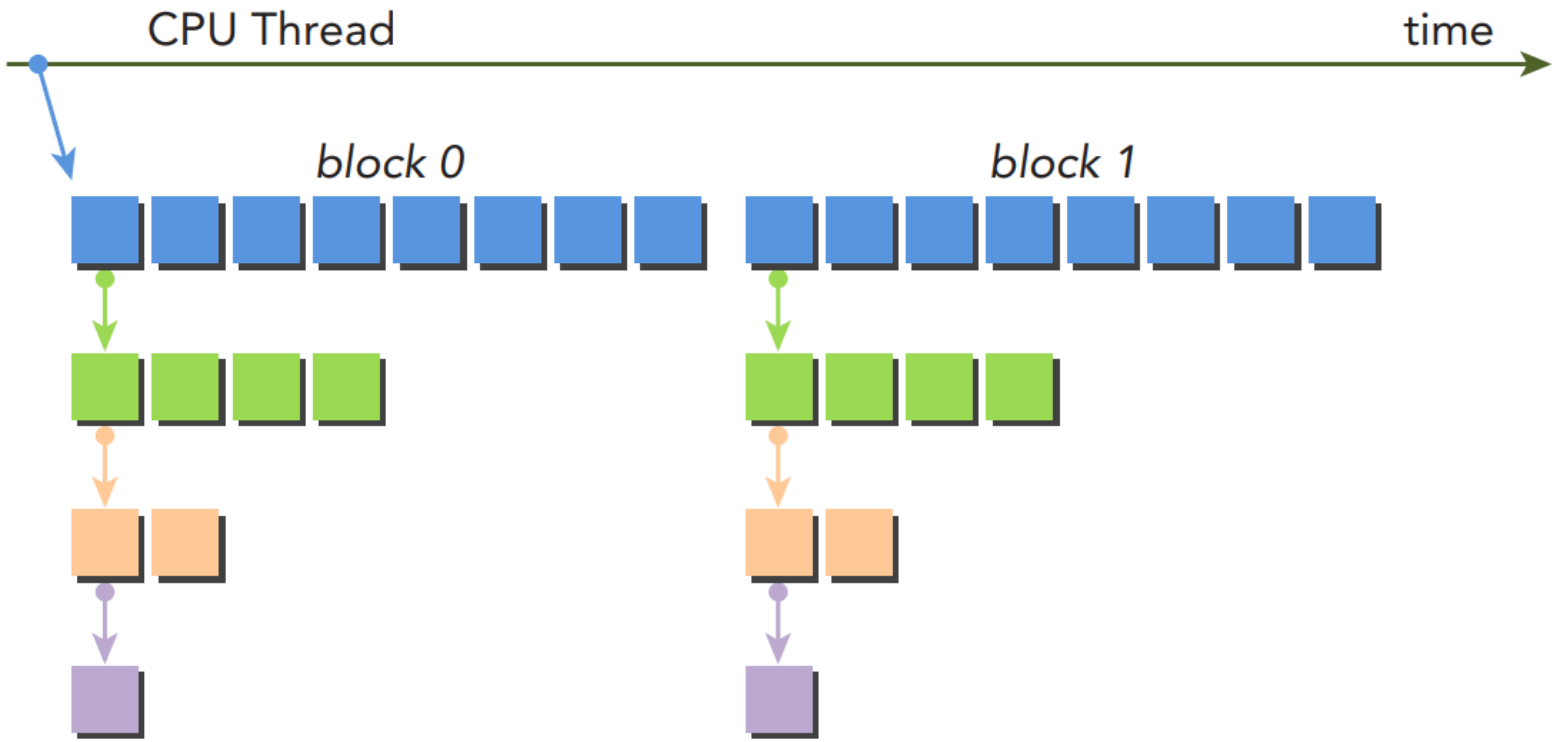
- Kernels invoked through dynamic parallelism cannot be launched on physically separate devices.

- The maximum nesting depth of dynamic parallelism is limited to 24
  - but in reality most kernels will be limited by the amount of memory required by the device runtime system at each new level

6.3 Nested Reduction

Nested Reduction

- **Approach 1:** The original grid containing many blocks, but all nested child grids holding only one block which is invoked by thread 0 of its parent.



```
__global__ void gpuRecursiveReduce (int *g_idata, int *g_odata,
    unsigned int isize) {
  // set thread ID
  unsigned int tid = threadIdx.x;

  // convert global data pointer to the local pointer of thi
  int *idata = g_idata + blockIdx.x*blockDim.x;
  int *odata = &g_odata[blockIdx.x];

  // stop condition
  if (isize == 2 && tid == 0) {
    g_odata[blockIdx.x] = idata[0]+idata[1];
    return;
  }

  // nested invocation
  int istride = isize>>1;
  if(istride > 1 && tid < istride) {
    // in place reduction
    idata[tid] += idata[tid + istride];
  }

  // sync at block level
  __syncthreads();

  // nested invocation to generate child grids
  if(tid==0) {
    gpuRecursiveReduce <<<1, istride>>>(idata,odata,istride);

    // sync all child grids launched in this block
    cudaDeviceSynchronize();
  }

  // sync at block level again
  __syncthreads();
```

if the stop condition is met (that is, if this is a leaf in the nested execution tree), results are copied back to global memory, and control is immediately returned back to the parent kernel.

If this is not a leaf kernel, the size of the local reduction is calculated and half of the threads perform an in-place reduction.

After the in-place reduction is complete, the block is synchronized to ensure all partial sums are computed.

thread 0 generates a child grid with one thread block and half as many threads as the current block

After the child grid is invoked, a barrier point is set for all child grids.

Nested Reduction

- Compared with the flat kernel using the neighbor-paired approach, nested kernel is unacceptably slow

```
./nestedReduce starting reduction at device 0: Tesla K40c
  array 1048576 grid 2048 block 512
cpu reduce      elapsed 0.000689 sec cpu_sum: 1048576
gpu Neighbored  elapsed 0.000532 sec gpu_sum: 1048576<<<grid 2048 block 512>>>
gpu nested      elapsed 0.172036 sec gpu_sum: 1048576<<<grid 2048 block 512>>>
```

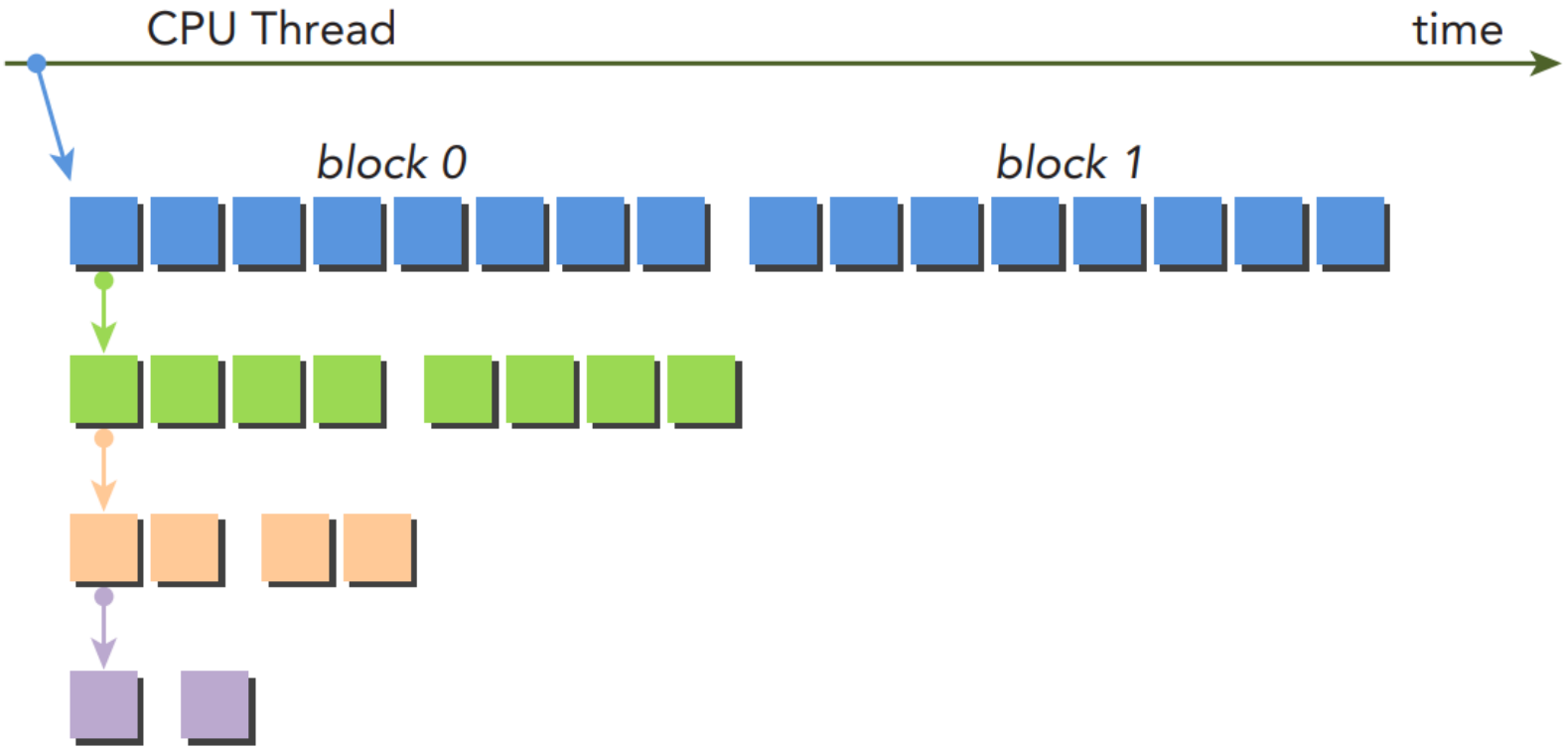
Such a large amount of kernel invocation and synchronization is likely the main cause for such poor kernel performance.

nestedReduce.cu



Nested Reduction

- **Approach 2:** The first thread in the first block of a grid invokes the child grids for each nested step.



Nested Reduction

```
__global__ void gpuRecursiveReduce2(int *g_idata, int *g_odata, int iStride,
    int const iDim) {
   // convert global data pointer to the local pointer of this block
   int *idata = g_idata + blockIdx.x*iDim;

   // stop condition
   if (iStride == 1 && threadIdx.x == 0) {
      g_odata[blockIdx.x] = idata[0]+idata[1];
      return;
   }

   // in place reduction
   idata[threadIdx.x] += idata[threadIdx.x + iStride];

   // nested invocation to generate child grids
   if(threadIdx.x == 0 && blockIdx.x == 0) {
      gpuRecursiveReduce2 <<<gridDim.x,iStride/2>>>(
          g_idata,g_odata,iStride/2,iDim);
   }
}
```

- all idle threads are removed from each kernel launch
- free half of the compute resources consumed by the first kernel so as to allow more thread blocks to become active.

Nested Reduction

- Recursive reduction kernel is again much faster than the first two implementations
  - likely due to decreased overhead from fewer child grid invocations.

```
./nestedReduce2 starting reduction at device 0: Tesla K40c
array 1048576 grid 2048 block 512
cpu reduce      elapsed 0.000689 sec cpu_sum: 1048576
gpu Neighbored  elapsed 0.000532 sec gpu_sum: 1048576<<<grid 2048 block 512>>>
gpu nested      elapsed 0.172036 sec gpu_sum: 1048576<<<grid 2048 block 512>>>
gpu nestedNosyn elapsed 0.059125 sec gpu_sum: 1048576<<<grid 2048 block 512>>>
gpu nested2     elapsed 0.000797 sec gpu_sum: 1048576<<<grid 2048 block 512>>>
```

nestedReduce2.cu



Summary

Summary

- You examined kernel execution from the hardware point-of-view.
- The two most distinguished features of the CUDA execution model on GPU devices are:
  - Threads are executed in warps in SIMT fashion.
  - Hardware resources are partitioned among blocks and threads.
- Grid and block heuristics play a very important role in optimizing kernel performance for different platforms.
- Dynamic parallelism gives you the ability to create new work directly from the device.